KHÓA HỌC CHO NGƯỜI MỚI · 150 SLIDE

Drone ESP32 hoạt động như thế nào

Linh kiện, thiết kế firmware và mã nguồn — từng dòng, bằng lời đơn giản.

Bo mạch XW_Z1 V2.0 | ESP32 + MPU6500 | Esp32_drone.ino | Ứng dụng Android · iOS

CHÀO MỪNG

Sau khóa học, bạn sẽ làm được gì

Giải thích cách bay

Vì sao bốn cánh quạt có thể bay lên, nghiêng và xoay — và vì sao nó rơi nếu không có máy tính.

Gọi tên mọi linh kiện

ESP32, cảm biến chuyển động, mạch lái động cơ, đo pin — và từng chân làm gì.

Đọc được firmware

Theo một vòng lặp của mã từ lúc đọc cảm biến đến lúc ra lệnh động cơ, 250 lần mỗi giây.

Sửa đổi an toàn

Biết có những lớp an toàn nào, chạy được bài kiểm thử, và sửa một chỗ nhỏ mà không làm hỏng chúng.

CHÀO MỪNG

Chiếc drone trong nháy mắt

4

động cơ chổi than 8520, cấp điện thẳng từ pin qua MOSFET

1

cảm biến chuyển động (MPU6500). Không có cảm biến áp suất, GPS hay la bàn

250

lần mỗi giây vòng điều khiển đọc, quyết định và đặt động cơ

Một chip ESP32 làm tất cả: điều khiển bay, Wi-Fi và Bluetooth tới điện thoại, và đầu vào máy thu vô tuyến SBUS.

CHÀO MỪNG

Bản đồ khóa học: 12 phần

#	Phần	#	Phần
1	Chào mừng	7	Bộ trộn và động cơ
2	Quadcopter bay như thế nào	8	Đầu vào: SBUS và CRTP
3	Các linh kiện phần cứng	9	Chế độ bay và an toàn
4	Kiến trúc firmware	10	Dẫn đường
5	Đọc cảm biến chuyển động	11	Kiểm thử và quy trình
6	Điều khiển PID	12	Tổng kết

Phần 2–3 không cần mã. Từ phần 4, mỗi ý tưởng đều được chỉ ra trong firmware thật.

CHÀO MỪNG

Cách đọc các slide mã nguồn

```
// chú thích giải thích dòng bên dưới
#define LOOP_TIME_US  4000

void loop() {
  mpuReadRaw();  // đọc cảm biến
}
```

Xám = chú thích. Trình biên dịch bỏ qua; nó dành cho người đọc.

Cam = từ khóa C++ như *void*, *if*, *float*.

Trắng = chính đoạn mã. Tên lấy từ file thật.

Mã được rút gọn bằng “…” khi một phần không cần cho ý tưởng. Chân trang cho biết nó nằm ở đâu.

CHÀO MỪNG

Trước khi chạm vào drone: ba quy tắc

1. **Tháo cánh quạt** cho mọi lần thử trên bàn. Cánh đang quay có thể cắt vào da.

2. **Biết cách dừng.** CẮT ĐỘNG CƠ trong ứng dụng, hoặc thao tác disarm trên tay điều khiển, trước khi arm.

3. **Mỗi lần chỉ đổi một thứ,** rồi thử. Đổi hai thứ cùng lúc sẽ không biết cái nào gây lỗi.

02

Quadcopter bay như thế nào

Bốn cánh quạt, ba trục, và vì sao nó không thể bay nếu thiếu máy tính.

CÁCH BAY

Quadcopter là bốn chiếc quạt thổi không khí xuống

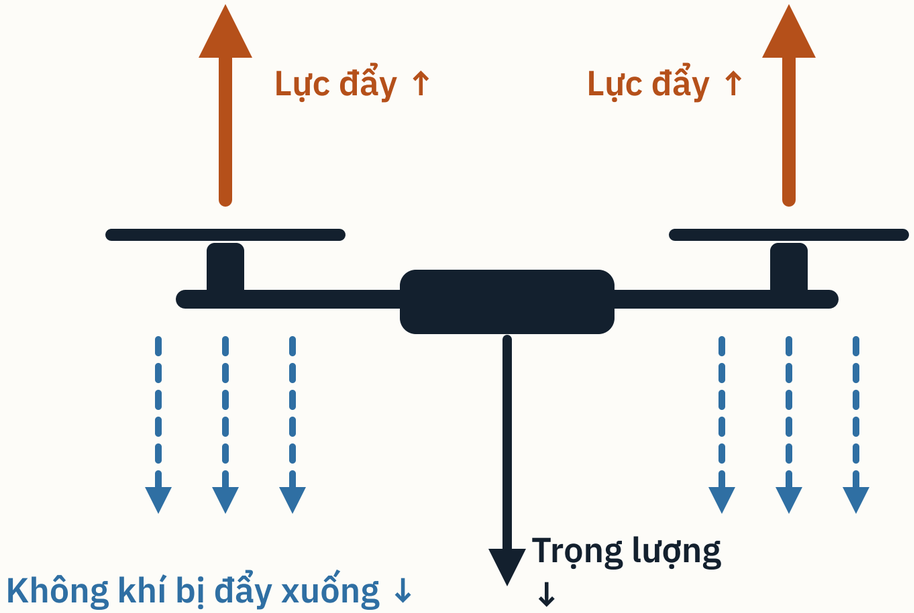
Mỗi cánh hất không khí xuống. Không khí đẩy cánh lên với lực bằng như vậy: lực đẩy.

Cánh quay nhanh hơn → nhiều không khí hơn mỗi giây → góc đó có nhiều lực đẩy hơn.

Không có cánh lái: khác máy bay, mọi chuyển động đến từ việc đổi tốc độ bốn động cơ.

VÍ DỤ

Chèo thuyền: mái chèo đẩy nước ra sau, thuyền tiến lên.



CÁCH BAY

Lực đẩy so với trọng lượng quyết định lên hay xuống

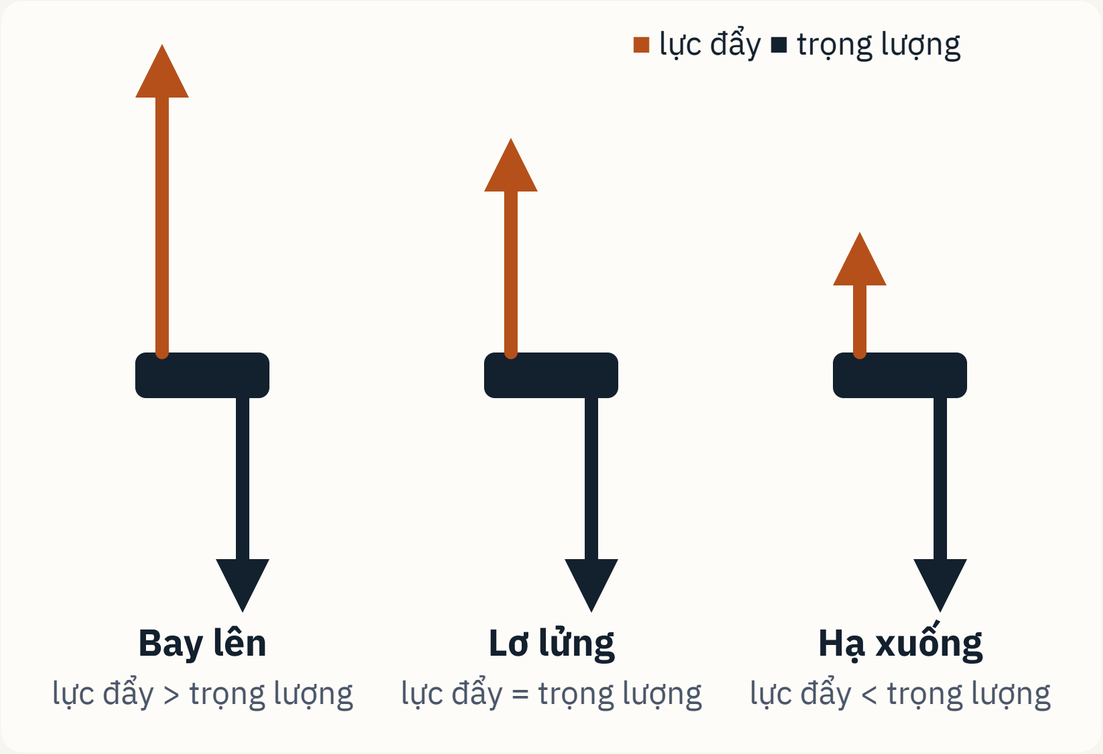
Lên hay xuống là cuộc kéo co giữa lực đẩy và trọng lượng.

Tăng ga → bay lên. Ga ở giữa → **lơ lửng**. Giảm ga → hạ xuống.

Khi pin yếu dần, cùng mức ga cho ít lực đẩy hơn — nên giữ nguyên mức ga thì drone từ từ hạ.

VÍ DỤ

Drone 40 g cần khoảng 40 g lực đẩy để lơ lửng: mỗi động cơ khoảng 10 g (số minh họa).



CÁCH BAY

Ba cách xoay: roll, pitch, yaw

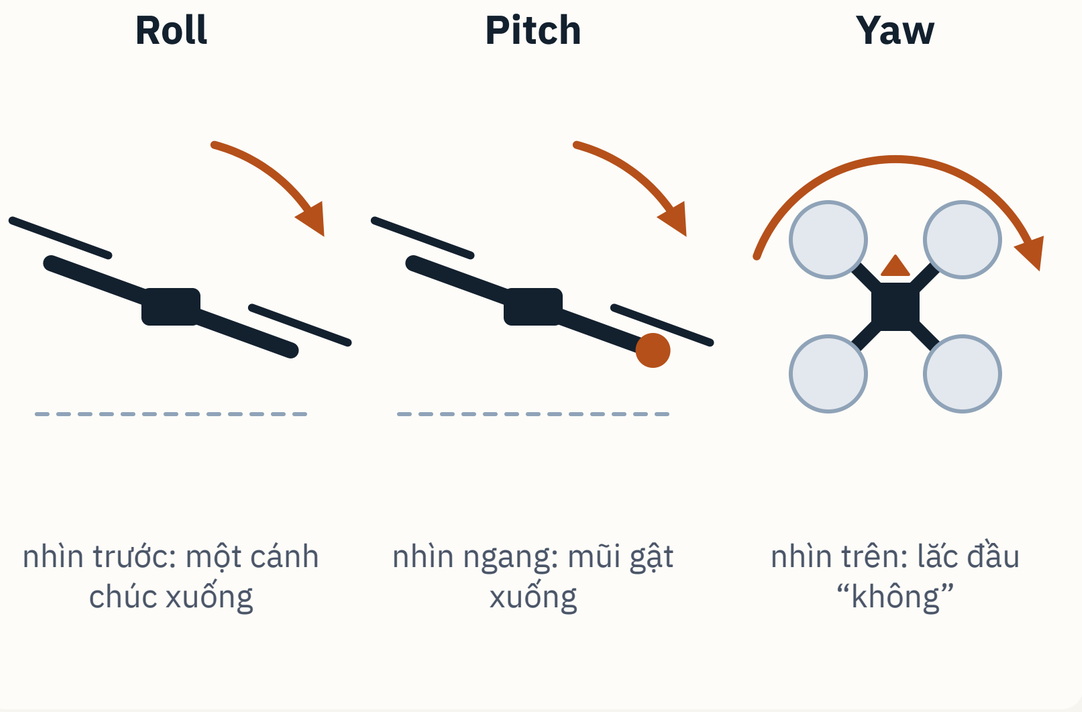
Ba cách xoay, cộng thêm ga.

Roll làm drone đi ngang.

Pitch làm drone tiến hoặc lùi.

Yaw đổi hướng mũi.

Cộng thêm **ga** để lên xuống: bốn lệnh điều khiển, và một bộ PID cho mỗi phép xoay.



CÁCH BAY

Hai cánh quay một chiều, hai cánh quay chiều kia

Cánh đang quay vặn drone theo chiều ngược lại. Trộn chiều quay sẽ triệt tiêu lực vặn.

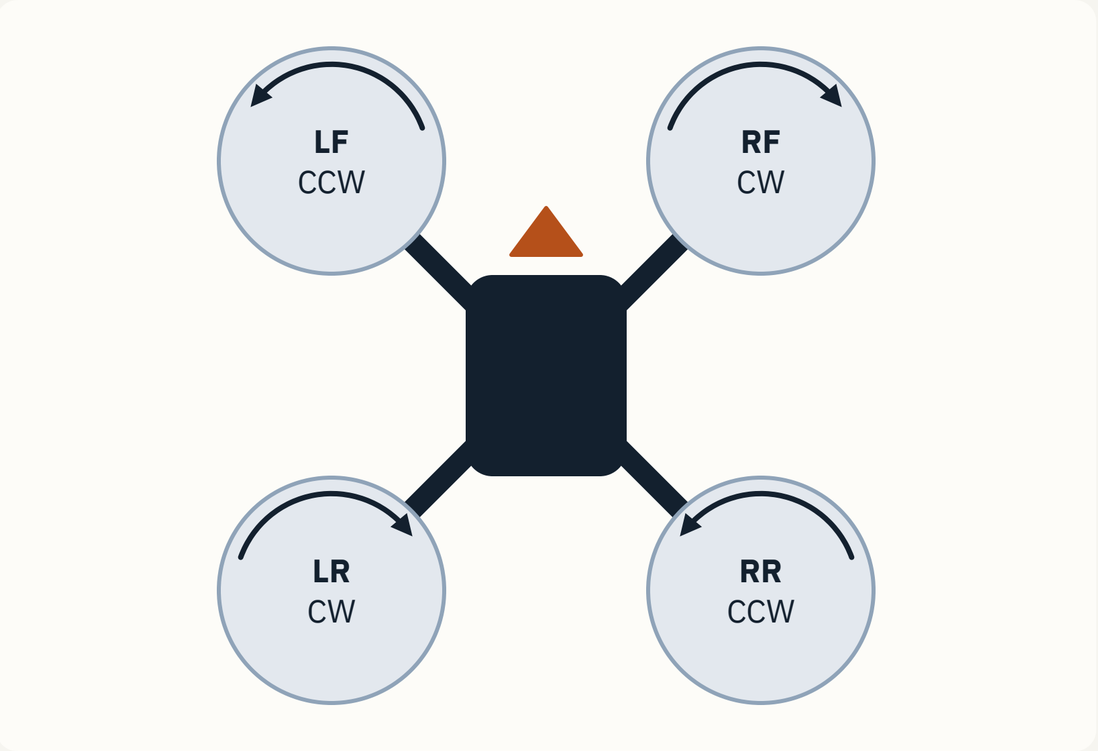
Thuận chiều (CW): trước-phải (RF) và sau-trái (LR).

Ngược chiều (CCW): sau-phải (RR) và trước-trái (LF).

Mỗi cặp nằm trên một đường chéo nên lực vặn triệt tiêu và drone cân bằng.

VÍ DỤ

Hai quạt trần treo lỏng, quay ngược chiều nhau, sẽ triệt tiêu lực vặn của nhau.



CÁCH BAY

Roll: cho một bên đẩy mạnh hơn

Để roll sang phải, động cơ trái tăng tốc và động cơ phải giảm tốc.

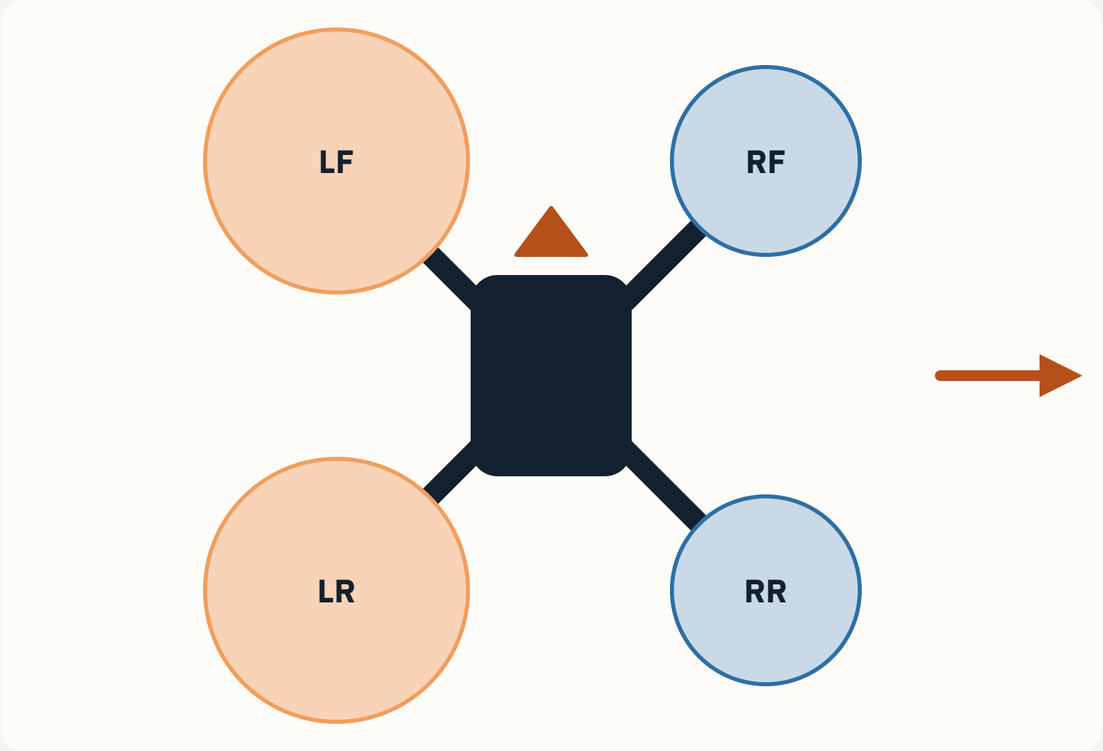
Cam, to hơn = nhanh hơn: LF và LR đẩy mạnh hơn, bên trái nâng lên.

Xanh, nhỏ hơn = chậm hơn: RF và RR đẩy yếu hơn, bên phải hạ xuống.

Kết quả: drone nghiêng phải và trượt sang phải →

VÍ DỤ

Lơ lửng 50% cả bốn. Roll phải: LF, LR → 55%; RF, RR → 45%. Trung bình vẫn 50%.



CÁCH BAY

Pitch: trước so với sau

Muốn bay tới, đẩy mạnh hơn ở phía sau.

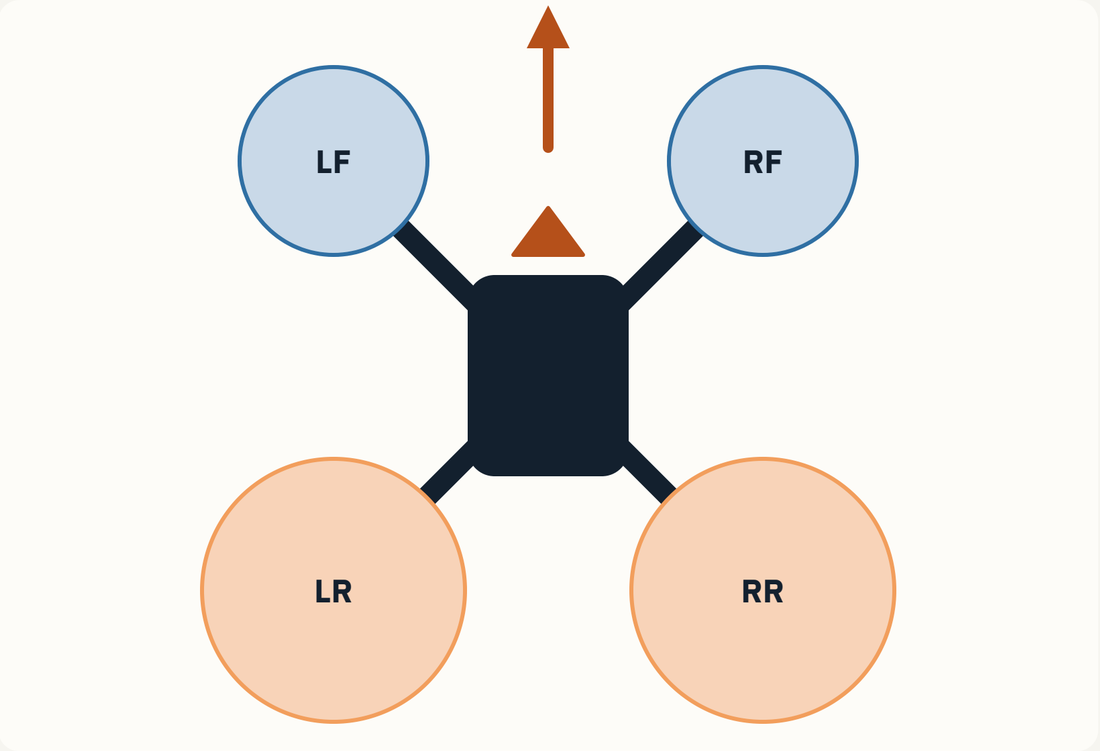
Sau (RR, LR) nhanh hơn → đuôi nâng lên.

Trước (RF, LF) chậm hơn → mũi hạ xuống.

Lực đẩy giờ hơi hướng về trước, nên drone tiến lên ↑

VÍ DỤ

Ngả người về trước trên xe đạp để bắt đầu lăn bánh: nghiêng trước, rồi mới đi.



CÁCH BAY

Yaw: dùng lực vặn một cách có chủ ý

Muốn xoay, dùng lực vặn một cách có chủ ý.

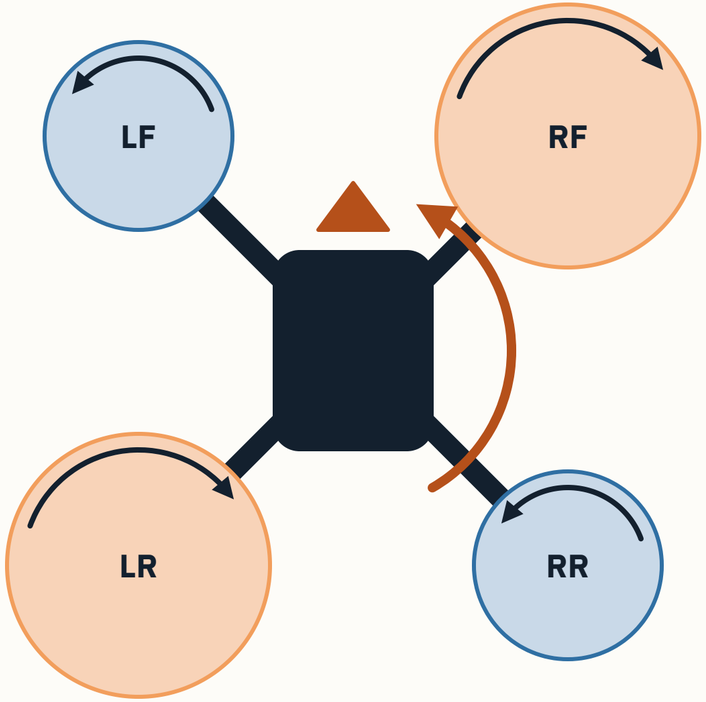
Tăng tốc **cặp CW (RF, LR)** và giảm tốc **cặp CCW (RR, LF)**.

Lực vặn không còn triệt tiêu: thân xoay theo chiều ngược lại (mũi tên cam).

Tổng lực đẩy không đổi, nên giữ độ cao.

VÍ DỤ

Ngồi ghế xoay, vung tay về một phía: người bạn xoay về phía kia.



CÁCH BAY

Tóm tắt: động cơ nào thay đổi cho mỗi động tác

Động tác	Nhanh hơn	Chậm hơn	Giữ nguyên
Bay lên	cả bốn	—	tư thế
Roll sang phải	LF, LR	RF, RR	tổng lực đẩy
Pitch tới (chúc mũi)	RR, LR	RF, LF	tổng lực đẩy
Yaw (cặp CW nhanh hơn)	RF, LR	RR, LF	tổng lực đẩy

Mọi động tác đều là “thêm chỗ này, bớt chỗ kia”. Chỉ bay lên mới đổi cả bốn như nhau.

Vì sao quadcopter không thể tự bay

Quadcopter không ổn định: một độ nghiêng nhỏ lớn dần chưa tới một giây.

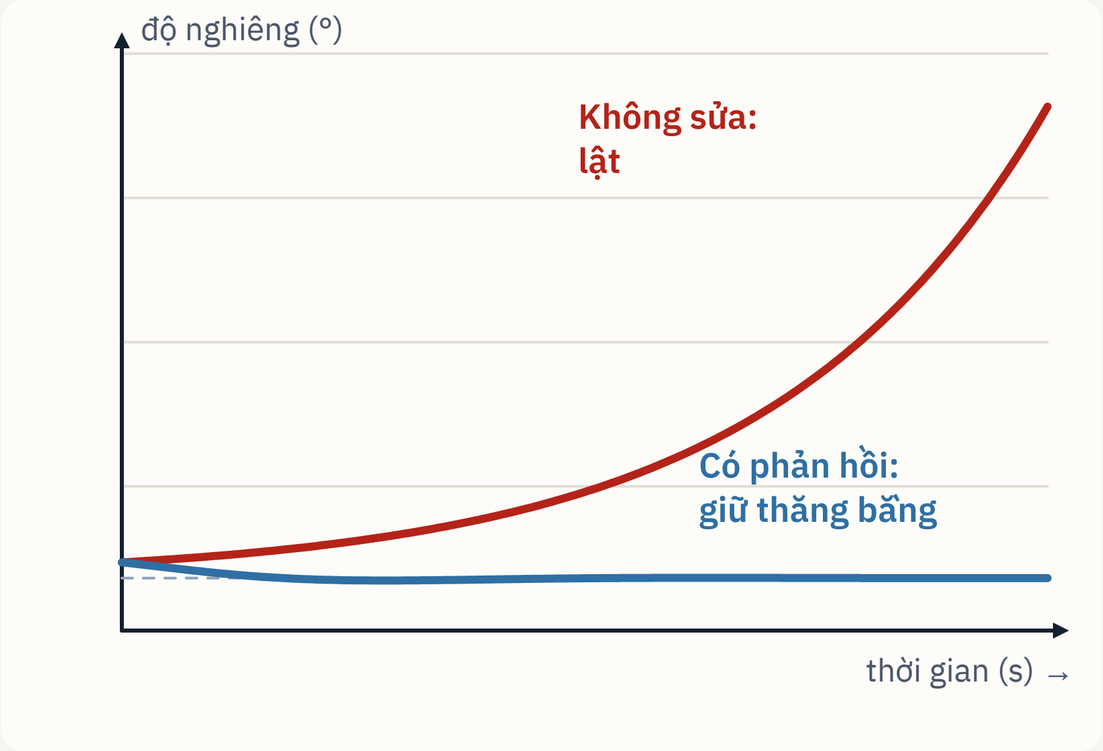
Động cơ không bao giờ bằng nhau: cùng mức điều khiển, lực đẩy hơi khác.

Không khí chuyển động: gió hay luồng gió của chính drone đẩy một bên.

Nghiêng tự khuếch đại: drone nghiêng thì trượt, trượt nhanh dần, rồi lật.

VÍ DỤ

Giữ thăng bằng cây chổi trên tay: ngừng chỉnh là nó đổ. (Đường cong là minh họa.)



CÁCH BAY

Cách sửa là phản hồi (feedback)

Ví dụ: giữ xe trong làn đường. Bạn thấy xe lệch, so với giữa làn, rồi xoay vô-lăng một chút — lặp đi lặp lại.

1. **Đo** drone đang làm gì lúc này (cảm biến chuyển động).
2. **So sánh** với điều ta muốn (setpoint từ cần điều khiển hoặc ứng dụng).
3. **Sửa**: đổi động cơ một lượng dựa trên chênh lệch (sai số).
4. **Lặp lại** — đủ nhanh để sai số luôn nhỏ.

CÁCH BAY

Một vòng điều khiển, 250 lần mỗi giây

Bốn bước giống nhau, 250 lần mỗi giây.

Cảm nhận: đọc gyro và gia tốc kế qua SPI.

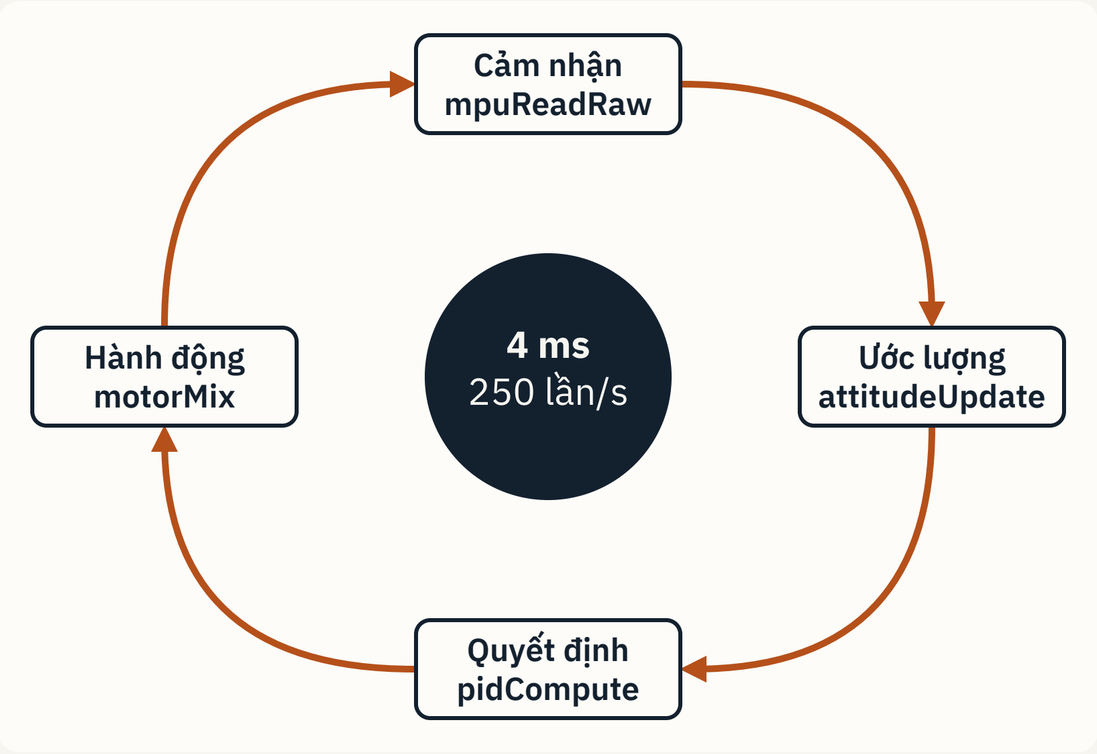
Ước lượng: tính các góc nghiêng.

Quyết định: PID so sánh điều muốn với điều đang có.

Hành động: bộ trộn đặt duty bốn động cơ — rồi chờ đủ 4 ms.

VÍ DỤ

Gyro báo roll 6 °/s → góc 2,1° → PID muốn 0 → động cơ trái +8, phải −8 → chờ.



CÁCH BAY

Vì sao 250 lần mỗi giây?

4 ms

mỗi vòng (LOOP_TIME_US 4000). Đủ ngắn để bắt kịp độ nghiêng trước khi nó lớn.

250 Hz

tốc độ lấy mẫu của cảm biến được đặt khớp: 1 kHz ÷ (1 + 3).

≈ 0,5°

ví dụ độ lệch giữa hai vòng — nhỏ, nên mỗi lần sửa cũng nhỏ.

Quy tắc nhớ: vòng điều khiển phải chạy nhanh hơn nhiều so với tốc độ drone có thể lật.

CÁCH BAY · HỎI TẠI SAO

Kiểm tra hiểu biết

Vì sao các cánh quạt quay CW và CCW xen kẽ?

Gợi ý: nghĩ về lực vặn mỗi cánh tạo lên thân.

Vì sao quadcopter chỉ đi ngang được bằng cách nghiêng?

Gợi ý: lực đẩy hướng về đâu?

Vì sao yaw cần hệ số lớn hơn roll?

Gợi ý: mô-men từ cánh nhỏ rất yếu.

Vì sao vòng lặp phải nhanh?

Gợi ý: một độ nghiêng nhỏ lớn lên nhanh thế nào?

03

Các linh kiện phần cứng

Mọi linh kiện trên bo mạch, chúng làm gì, và firmware nói chuyện với chúng thế nào.

PHẦN CỨNG

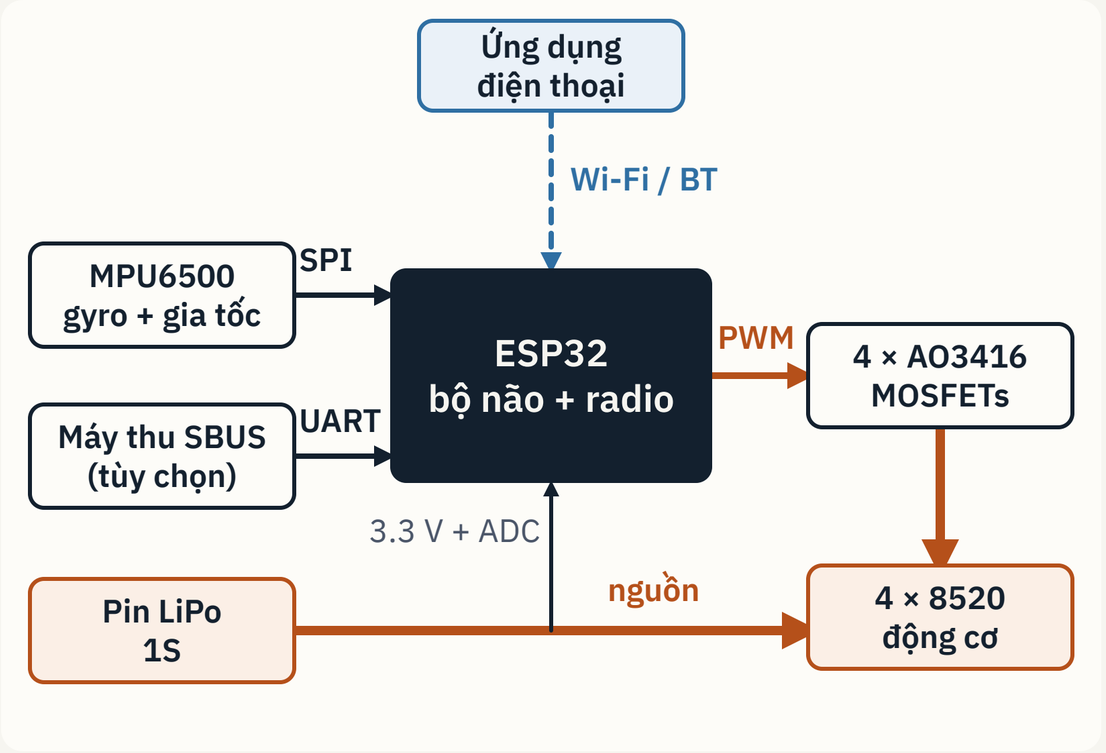
Bo mạch trong một hình

Một chip ở giữa; mọi thứ khác cắm vào nó.

Đầu vào (trái): cảm biến chuyển động qua SPI, máy thu RC tùy chọn qua UART, điện áp pin qua ADC.

Đầu ra (phải): PWM tới bốn công tắc MOSFET, lái bốn động cơ.

Radio (trên): điện thoại nói chuyện với ESP32 qua Wi-Fi hoặc Bluetooth.



PHẦN CỨNG

ESP32: bộ não

Hai nhân, tới 240 MHz

Dư sức cho vòng 250 Hz với phép tính số thực.

Wi-Fi và Bluetooth có sẵn

Điện thoại kết nối trực tiếp — không cần chip radio riêng.

Khối phần cứng hỗ trợ

SPI, UART, PWM LEDC và ADC làm việc định thời bằng phần cứng.

VÍ DỤ

Như một chiếc điện thoại kiêm luôn bộ phát Wi-Fi: vừa tính toán vừa liên lạc, không cần hộp thêm.

PHẦN CỨNG

MPU6500: cảm biến chuyển động

Một chip nhỏ chứa hai cảm biến, mỗi cảm biến đo ba trục.

Con quay hồi chuyển (gyro)

Drone xoay nhanh bao nhiêu, theo độ/giây. Dải đặt ±500 °/s.

Gia tốc kế

Lực đẩy mà nó cảm nhận, gồm cả trọng lực. Dải đặt ±8 g.

Kết nối SPI

ESP32 đọc 14 byte một lượt mỗi vòng.

PHẦN CỨNG

Gyro và gia tốc kế: mỗi cái sửa cho cái kia

Mỗi cảm biến sai theo một cách khác nhau.

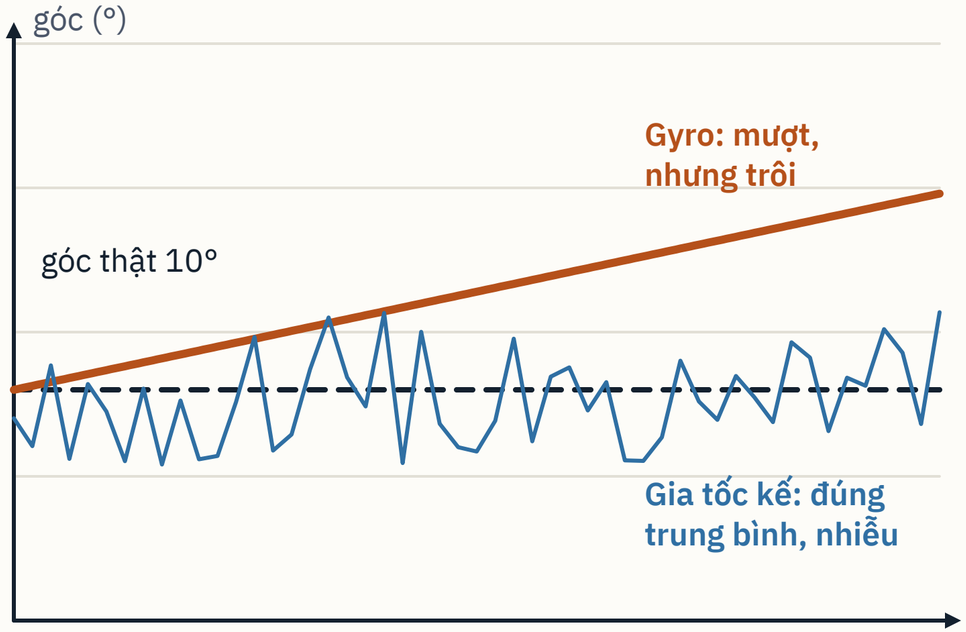
Gyro: nhanh và mượt, nhưng độ lệch nhỏ cộng dồn — góc bị trôi.

Gia tốc kế: biết "phía dưới" ở đâu và không trôi, nhưng rung làm nó nhiễu.

Phần 5 kết hợp chúng để mỗi cái bù điểm yếu của cái kia.

VÍ DỤ

Gyro = đồng hồ bấm giờ (chính xác, trôi). Gia tốc kế = mặt trời (luôn đúng, khó đọc khi trời mây).



PHẦN CỨNG

SPI: bốn dây nối tới cảm biến

Bốn dây, mỗi dây một chiều.

SCK: xung nhịp — ESP32 đặt tốc độ.

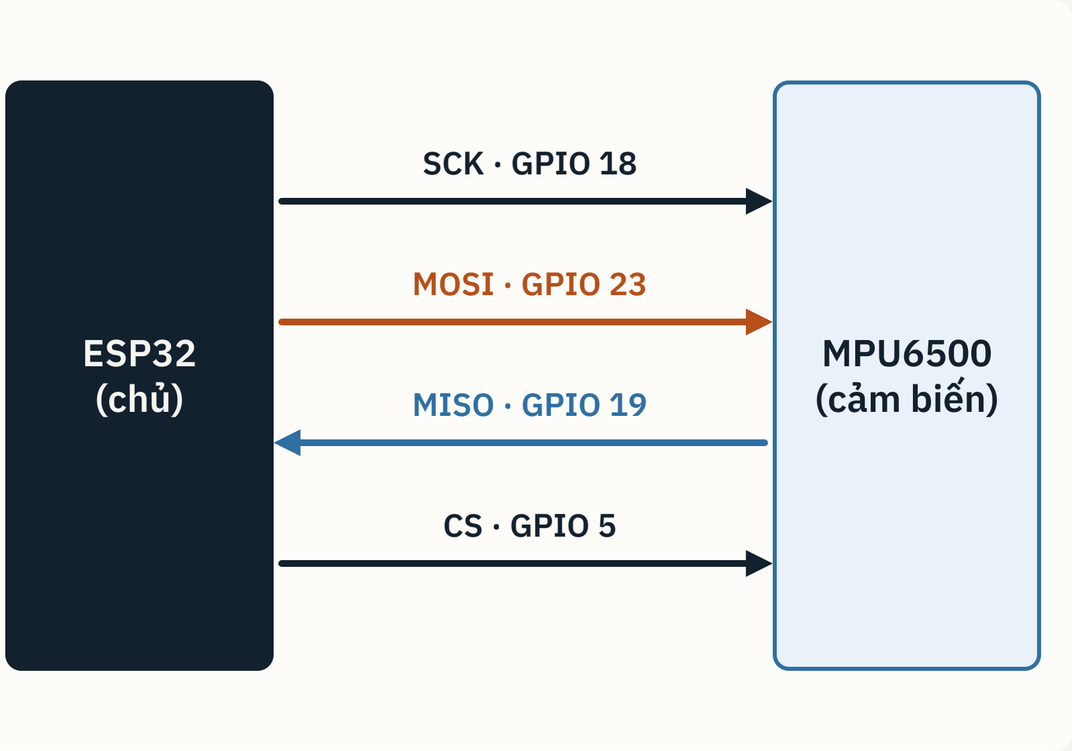
MOSI: dữ liệu ra cảm biến. **MISO**: dữ liệu về.

CS: LOW nghĩa là “tôi đang nói với bạn”.

Firmware chạy SPI ở 1 MHz, chế độ 0.

VÍ DỤ

Đọc 14 byte ở 1 MHz mất khoảng 0,12 ms — 3% của vòng 4 ms.



PHẦN CỨNG

SBUS: máy thu RC tùy chọn

Một máy thu vô tuyến mô hình có thể cắm vào GPIO 4 để bay bằng tay điều khiển thật.

Một dây tín hiệu

16 kênh gói trong một khung 25 byte.

Chuẩn nối tiếp lạ

100000 baud, 8 bit dữ liệu, parity chẵn, 2 bit stop, đảo mức.

Lưới an toàn

SBUS luôn bay ở chế độ rate thủ công. Phần 8 giải mã nó.

VÍ DỤ

Serial2.begin(100000, SERIAL_8E2, PIN_SBUS_RX, -1, true); — chữ true cuối cùng bật chế độ đảo.

PHẦN CỨNG

MOSFET AO3416: công tắc động cơ

Một chân GPIO không thể cấp điện cho động cơ. MOSFET là công tắc điện tử làm được việc đó.

Tín hiệu nhỏ vào

Chân ESP32 điều khiển cực gate bằng 3,3 V và gần như không tốn dòng.

Dòng lớn đi qua

MOSFET cho dòng từ pin chạy qua động cơ.

Nhanh

Có thể bật tắt 20.000 lần mỗi giây.

VÍ DỤ

Như một công tắc đèn bạn bật bằng một ngón tay mà điều khiển được cả chiếc đèn lớn.

PHẦN CỨNG

Động cơ chổi than không lõi 8520

Tên chính là kích thước: rộng 8,5 mm, dài 20 mm.

Chổi than

Quay nhanh hơn khi điện áp trung bình cao hơn. Không cần ESC.

Không lõi

Rotor nhẹ nên đổi tốc độ nhanh — tốt cho điều khiển.

Lái bằng độ rộng xung

Firmware đặt tỷ lệ thời gian MOSFET mở trong mỗi chu kỳ.

PHẦN CỨNG

PWM: bật tắt nhanh tạo ra giá trị trung bình

Bật-tắt, rất nhanh. Động cơ cảm nhận giá trị trung bình.

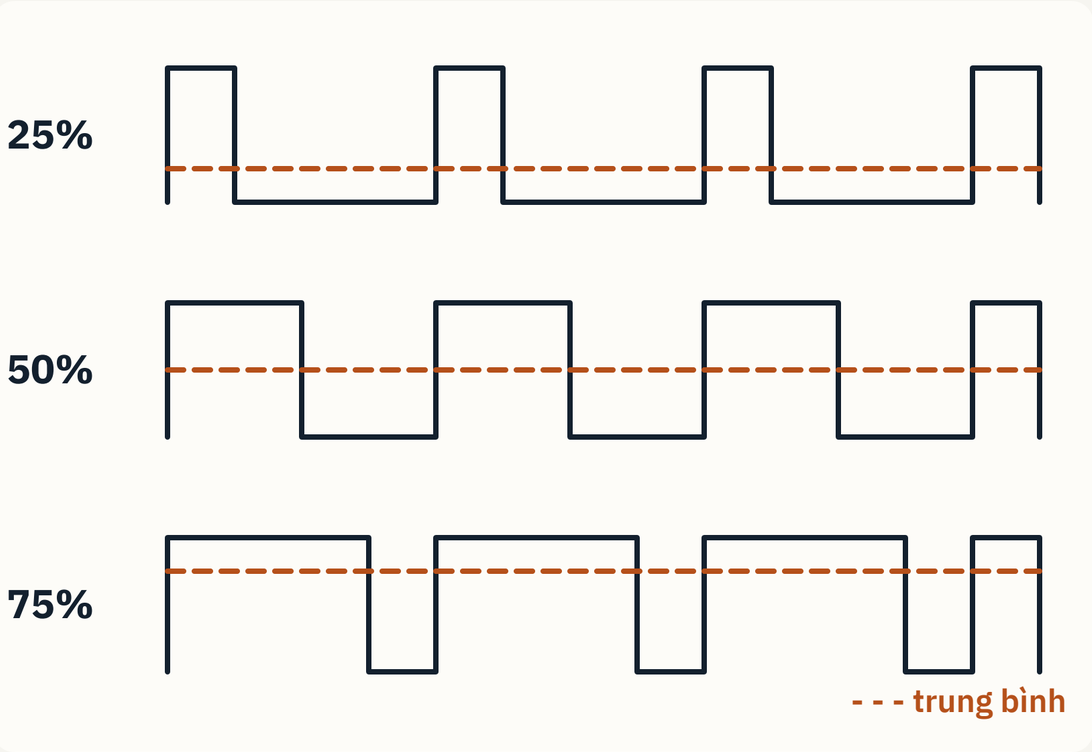
Duty: tỷ lệ thời gian công tắc BẬT trong mỗi chu kỳ.

20 kHz: một chu kỳ mỗi 50 µs — quá nhanh để động cơ nhận ra.

10 bit: duty là số từ 0 đến 1023.

VÍ DỤ

Pin 4,0 V, duty 256 trên 1023 (25%) → động cơ cảm nhận khoảng 1,0 V.



PHẦN CỨNG

Pin: một cell LiPo (1S)

Một cell LiPo (1S): điện áp chính là đồng hồ nhiên liệu.

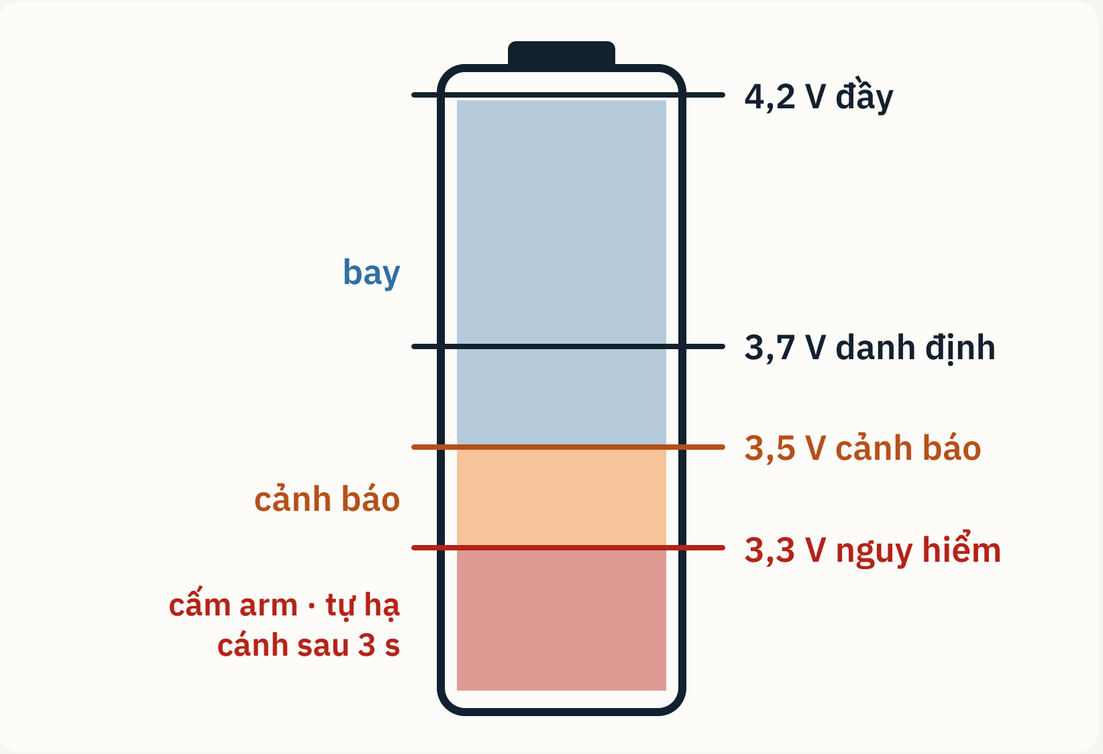
Cell đầy là **4,2 V**.

Dưới **3,5 V** firmware cảnh báo (VBAT_CELL_WARN).

Dưới **3,3 V** không cho arm, và tự hạ cánh nếu ở mức đó 3 s (VBAT_CELL_CRIT).

VÍ DỤ

Lực đẩy cũng giảm theo điện áp — vì vậy độ cao lơ lửng tụt dần trong chuyến bay.



PHẦN CỨNG

Đo pin: cầu chia điện áp

Hai điện trở hạ điện áp pin xuống mức an toàn.

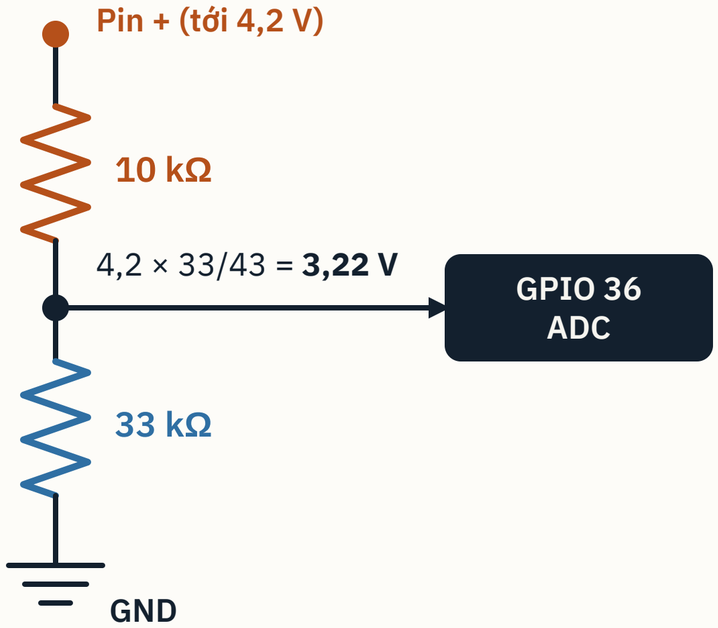
Chân ADC đọc tới khoảng 3,3 V, nhưng cell đầy là 4,2 V.

Điện áp chân = pin × 33 ÷ 43. Firmware nhân ngược lại **43 ÷ 33** (VBAT_DIVIDER).

Đổi điện trở → phải đổi cả VBAT_DIVIDER.

VÍ DỤ

Chân đọc 3,00 V → pin = 3,00 × 43 ÷ 33 = 3,91 V.



PHẦN CỨNG

Đọc và làm mượt điện áp pin

```
uint32_t mv = analogReadMilliVolts(PIN_VBAT);
float sample = (mv / 1000.0f) * VBAT_DIVIDER;
if (vbat ⩽ 0.0f) vbat = sample;          // first
read
else vbat += VBAT_FILTER_ALPHA * (sample - vbat);
```

analogReadMilliVolts dùng hiệu chuẩn có sẵn từ nhà máy.

Alpha 0,02: mỗi vòng tiến 2% về giá trị mới — hằng số thời gian khoảng 0,2 s.

Vì sao làm mượt? Một lần đọc thì nhiễu và sụt theo mỗi cú tăng ga.

VÍ DỤ

Cũ 3,90 V, một mẫu nhiễu 3,60 V → mới 3,90 + 0,02 × (−0,30) = 3,894 V. Một lần đọc xấu gần như không ảnh hưởng.

PHẦN CỨNG

Sơ đồ chân

Chân	Dùng cho	Chân	Dùng cho
GPIO 15	Động cơ RF (trước-phải)	GPIO 18	SPI SCK
GPIO 13	Động cơ RR (sau-phải)	GPIO 19	SPI MISO
GPIO 12	Động cơ LR (sau-trái)	GPIO 23	SPI MOSI
GPIO 14	Động cơ LF (trước-trái)	GPIO 5	Chọn chip MPU
GPIO 4	SBUS vào (UART2 RX)	GPIO 36	ADC đo pin

PHẦN CỨNG

Bố trí động cơ, nhìn từ trên xuống

Nhìn từ trên xuống, mũi hướng lên.

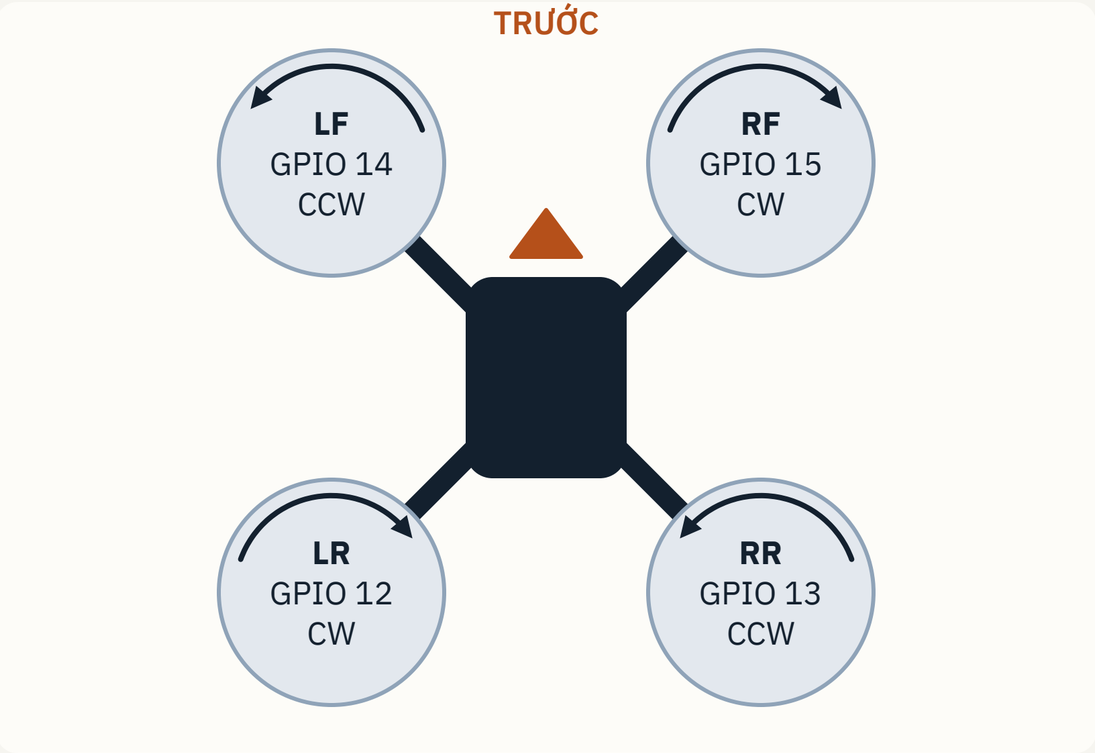
Khung X: mũi hướng vào giữa hai động cơ trước.

Đường chéo khớp nhau: RF và LR thuận chiều; LF và RR ngược chiều.

Tên tính theo góc nhìn của drone, nhìn về phía trước.

VÍ DỤ

Tháo cánh, arm, nghiêng nhẹ bằng tay: động cơ phía thấp phải tăng tốc.



PHẦN CỨNG

Nếu cánh quạt lắp ngược: YAW_DIRECTION

Một số bản lắp đặt cánh CW ở RR và LF. Khi đó mọi lệnh sửa yaw đều đẩy sai hướng.

Triệu chứng

Drone xoay ngày càng nhanh ngay khi rời mặt đất.

Cách sửa

Đổi #define YAW_DIRECTION từ 1 thành −1.

Vì sao được

Bộ trộn nhân đầu ra yaw với YAW_DIRECTION, đảo dấu cả bốn.

PHẦN CỨNG

Đường đi của nguồn điện

Một viên pin cấp ba nhánh. Độ dày đường = dòng điện.

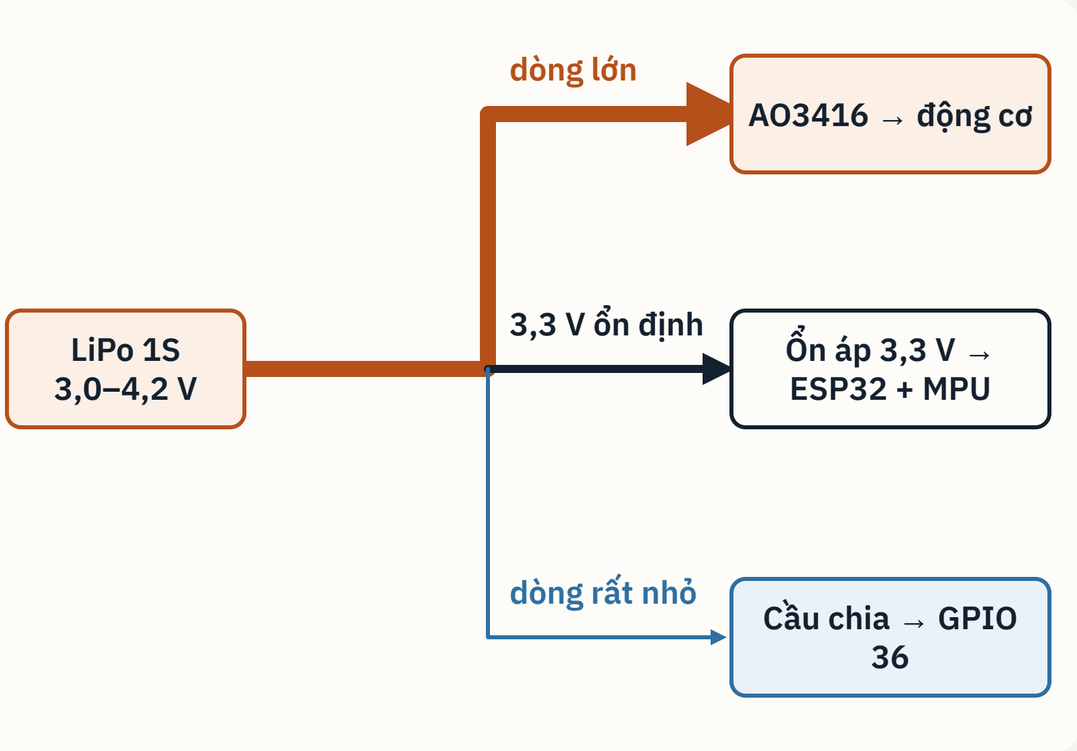
Động cơ lấy phần lớn dòng, thẳng từ pin.

Mạch logic cần 3,3 V ổn định, nên đi qua ổn áp.

Cầu chia giúp firmware theo dõi điện áp.

VÍ DỤ

Tăng ga hết cỡ: pin sụt 3,9 → 3,6 V trong chốc lát; ESP32 vẫn thấy 3,3 V ổn định.



PHẦN CỨNG

Những gì KHÔNG có trên bo mạch — và vì sao quan trọng

Không cảm biến áp suất

Không đo được độ cao. Hold giữ mức lực đẩy, không giữ độ cao.

Không la bàn

Hướng chỉ lấy từ gyro nên trôi dần.

Không GPS

Không biết vị trí. Follow dùng GPS điện thoại và ước tính vị trí.

Không cảm biến khoảng cách

Không thấy mặt đất. Hạ cánh là một đường dốc lực đẩy theo thời gian.

PHẦN CỨNG

Tóm tắt linh kiện

Linh kiện	Nhiệm vụ	Kết nối	Mã chính
ESP32	chạy mọi thứ	—	setup(), loop()
MPU6500	xoay + trọng lực	SPI	mpuInit(), mpuReadRaw()
Máy thu SBUS	cần RC (tùy chọn)	UART2 RX	sbusPoll(), sbusDecode()
AO3416 × 4	công tắc động cơ	PWM LEDC	motorInit(), motorWrite()
8520 × 4	lực đẩy	qua MOSFET	motorMix()
Cầu chia	điện áp pin	ADC GPIO 36	batteryUpdate()

PHẦN CỨNG · HỎI TẠI SAO

Kiểm tra hiểu biết

Vì sao chân GPIO cần MOSFET để chạy động cơ?

Gợi ý: so sánh dòng của chân với dòng của động cơ.

Vì sao PWM ở 20 kHz mà không phải 1 kHz?

Gợi ý: hãy lắng nghe động cơ.

Vì sao phải chia điện áp pin trước khi vào ADC?

Gợi ý: điện áp tối đa của ADC là bao nhiêu?

Vì sao drone không giữ được độ cao?

Gợi ý: cảm biến nào đo độ cao?

04

Kiến trúc firmware

Một sketch Arduino, một vòng lặp, và thứ tự mọi việc diễn ra.

KIẾN TRÚC

Một sketch, hai điểm vào

Mọi chương trình Arduino có cùng hình dạng. Esp32_drone.ino cũng vậy.

setup()

Chạy một lần khi bật nguồn. Khởi động phần cứng, nạp cài đặt đã lưu, hiệu chuẩn gyro.

loop()

Chạy mãi mãi, mỗi 4 ms một lần. Đọc, quyết định, điều khiển động cơ.

Không có tác vụ RTOS

Mã bay không tự tạo luồng riêng. Một vòng lặp, một thứ tự.

VÍ DỤ

Như một đầu bếp làm theo một tấm công thức từ trên xuống, lặp đi lặp lại, thay vì nhiều đầu bếp làm cùng lúc.

KIẾN TRÚC

setup() làm gì, theo thứ tự

1. **motorInit()** và **motorStop()** — tắt động cơ trước mọi thứ.
2. **mpuInit()** — nếu WHO_AM_I sai, in gợi ý kiểm tra dây và dừng mãi.
3. Đặt suy giảm ADC cho chân pin, rồi **sbusInit()**.
4. Mở NVS và **cfgLoad()** — hệ số PID và cài đặt bay đã lưu.
5. **linkInit()** (Wi-Fi / Bluetooth), rồi **mpuCalibrateGyro()** và đọc pin lần đầu.

KIẾN TRÚC

Bản đồ của sketch

Khu vực	Các hàm chính	Phần khóa học
#define cấu hình	chân, tần số, hệ số, giới hạn	3, 4
MPU6500	mpuInit, mpuReadRaw, mpuCalibrateGyro	5
SBUS	sbusInit, sbusDecode, sbusPoll	8
Truyền dẫn + CRTP	linkInit, linkPoll, crtpDispatch, các handler	8
Động cơ + pin	motorInit, usToDuty, motorWrite, batteryUpdate	3, 7
PID + tư thế	pidCompute, attitudeUpdate	5, 6
Dẫn đường	navUpdate, navSetpoints	10
Chế độ + arm	flightSetpoints, armUpdate, motorMix	6, 7, 9

KIẾN TRÚC

Trạng thái toàn cục: các biến dùng chung

Biến	Ý nghĩa	Ai ghi
armed	động cơ được phép quay	armUpdate, lệnh bay, failsafe
flight_mode, auto_state	MANUAL…RTH, cất/giữ/hạ cánh	lệnh port 7
gyro_roll / pitch / yaw	tốc độ xoay, °/s	attitudeUpdate
angle_roll_deg / pitch	góc nghiêng đã lọc, °	attitudeUpdate
sp_* (setpoint)	điều ta muốn ở vòng này	flightSetpoints
motor_rf … motor_lf	lệnh 1000–2000 cho từng động cơ	motorMix

KIẾN TRÚC

loop() trong mười hai dòng

```
sbusPoll();  linkPoll();              // 1. receive
// 2. decide who is in control, failsafe
if (!rx_ok && armed) { armed = false; motorStop(); }
armUpdate(now, use_crtp);             // 3. arm / disarm
mpuReadRaw();  attitudeUpdate();      // 4. sense
navUpdate(...);                       // 5. dead reckoning
flightSetpoints(now, use_crtp, sbus_ok);  // 6. want
roll_setpoint = sp_roll_rate - roll_level_adjust;
pidCompute();                         // 7. decide
motorMix(sp_throttle_us);             // 8. mix
motorWrite(motor_rf, motor_rr, motor_lr, motor_lf);
batteryUpdate(); logUpdate(); telemetry(now);
while (micros() - loop_timer < LOOP_TIME_US) { }
```

Đã rút gọn — loop thật có chú thích và các kiểm tra giữa các dòng.

Thứ tự quan trọng: kiểm tra an toàn chạy trước cảm biến và điều khiển, nên mất kết nối sẽ dừng động cơ ngay trong vòng đó.

KIẾN TRÚC

Dữ liệu chạy qua một vòng thế nào

Đầu vào
Cần SBUS hoặc CRTP từ điện thoại

→

Setpoint
flightSetpoints(): góc và tốc độ mục tiêu

→

Tự cân bằng
sai số góc → tốc độ mục tiêu

→

PID
sai số tốc độ → lệnh sửa

→

Bộ trộn
lệnh sửa → 4 động cơ

→

PWM
duty tới MOSFET

Dữ liệu cảm biến vào hai lần: góc ở bước tự cân bằng, tốc độ gyro ở PID.

VÍ DỤ

Cần sang phải → sp_roll_rate +50 °/s → gyro báo 0 → sai số −50 → đầu ra PID → động cơ phải/trái thay đổi → drone bắt đầu lăn.

KIẾN TRÚC

Nhịp cứng 250 Hz

```
// ---- hard 250 Hz pacing ----
while (micros() - loop_timer < LOOP_TIME_US) { /*
spin */ }
loop_timer = micros();
```

Chờ bận: CPU quay vòng cho tới khi đúng 4 ms kể từ vòng trước.

Vì sao cần đúng giờ? Gyro được tích phân với dt cố định 0,004 s. Vòng dài sai thì góc sai.

An toàn khi tràn số: phép trừ không dấu vẫn đúng khi micros() quay vòng sau ~71 phút.

VÍ DỤ

Vòng này làm việc mất 1,3 ms → chờ 2,7 ms. Vòng sau mất 2,1 ms → chờ 1,9 ms. Chu kỳ vẫn là 4 ms.

KIẾN TRÚC

Đầu vào: ai có thể lái drone

Ứng dụng điện thoại (CRTP)

Qua Wi-Fi UDP, Bluetooth Classic SPP hoặc BLE. Thắng khi kết nối còn sống.

Máy thu RC (SBUS)

Tùy chọn. Dùng khi điện thoại im lặng. Luôn ở chế độ rate thủ công.

Cổng serial

Cáp USB, để cài đặt trên bàn: link, sta, save, load, defaults, status.

VÍ DỤ

Điện thoại kết nối và đang gửi → điện thoại lái. Ứng dụng tắt 0,5 s → SBUS tiếp quản nếu có máy thu; nếu không, failsafe dừng động cơ.

KIẾN TRÚC

Đầu ra: firmware tạo ra gì

PWM động cơ

Bốn giá trị duty, mỗi 4 ms. Đầu ra duy nhất làm drone chuyển động.

Phản hồi + log CRTP

Xác nhận lệnh, giá trị tham số, và tối đa 3 luồng log về điện thoại.

Telemetry serial

Một dòng dễ đọc mỗi giây: VBAT, cần điều khiển, góc, động cơ.

KIẾN TRÚC

Cấu hình nằm trong các #define

#define	Giá trị	Thay đổi gì
LOOP_TIME_US	4000	chu kỳ vòng lặp (250 Hz)
PID_ROLL_KP / KI / KD	1,3 / 0,04 / 18	hệ số roll mặc định
MAX_TILT_DEG	25	góc nghiêng lệnh lớn nhất
LEVEL_SENSITIVITY	15	tự cân bằng kéo về mạnh cỡ nào
FAILSAFE_MS	500	im lặng bao lâu thì dừng động cơ
VBAT_CELL_CRIT	3,30	điện áp cấm arm và tự hạ cánh

KIẾN TRÚC

Lưu cài đặt: flash NVS

```
void cfgSave() {
  float g[9] = { pid_p_gain_roll, ... };   // 9
gains
  uint16_t f[3] = { hover_thrust, takeoff_ms,
land_ms };
  nvs.putBytes("pid",  g, sizeof(g));
  nvs.putBytes("fcfg", f, sizeof(f));
}
```

NVS = bộ nhớ không mất khi tắt nguồn: kho khóa–giá trị nhỏ trong flash ESP32.

Lưu số thực thô: đọc lại giống hệt từng byte — không làm tròn.

Bị từ chối khi đang arm: xóa flash làm cả hai nhân đứng vài mili giây.

VÍ DỤ

Chỉnh roll_kp thành 1,5 trên điện thoại, hạ cánh, nhấn Save → giá trị còn sau lần tắt nguồn tới.

KIẾN TRÚC

Các cấu hình kết nối điện thoại

#	Cấu hình	Phù hợp khi
0	Wi-Fi điểm phát (drone là hotspot)	mặc định; đơn giản nhất
1	Wi-Fi station (vào router nhà bạn)	điện thoại vẫn dùng Wi-Fi nhà
2	Bluetooth Classic SPP	điện thoại Android
3	Bluetooth Low Energy	iPhone (iOS không có SPP)
4	Wi-Fi điểm phát + BLE	hai đường vào cùng lúc
5	Wi-Fi station + BLE	router kèm BLE dự phòng

KIẾN TRÚC

Bốn nguyên tắc thiết kế bạn sẽ thấy khắp nơi

An toàn trước trong vòng lặp

Failsafe và arm chạy trước điều khiển, mỗi vòng.

Trung thực về giới hạn

Không có cảm biến áp suất → không giả độ cao. Chú thích nói rõ.

Lệnh idempotent

Ứng dụng gửi lại mỗi 500 ms, nên áp dụng một lệnh hai lần phải an toàn.

Kiểm thử ghim dòng dễ hỏng

run.sh báo lỗi nếu công thức đã kiểm thử bị đổi trong sketch.

KIẾN TRÚC · HỎI TẠI SAO

Kiểm tra hiểu biết

Vì sao setup() tắt động cơ đầu tiên?

Gợi ý: cực gate thả nổi có thể làm gì?

Vì sao dùng chờ bận thay vì delay()?

Gợi ý: chu kỳ thay đổi thế nào khi thời gian làm việc thay đổi?

Vì sao lệnh phải an toàn khi áp dụng hai lần?

Gợi ý: ứng dụng làm gì khi mất xác nhận?

Vì sao SAVE bị từ chối khi đang arm?

Gợi ý: xóa flash mất bao lâu?

05

Đọc cảm biến chuyển động

Từ các byte SPI thô đến một góc nghiêng đáng tin.

CẢM BIẾN

Đọc ghi một thanh ghi qua SPI

```
uint8_t mpuRead(uint8_t reg) {
  mpuSelect();                        // CS low
  SPI.transfer(reg | 0x80);           // bit7 = 1 → read
  uint8_t v = SPI.transfer(0x00);     // clock the answer out
  mpuDeselect();                      // CS high
  return v;
}
// write: reg & 0x7F (bit7 = 0), then the value
```

Byte đầu = địa chỉ thanh ghi. Bit cao nhất cho biết đọc (1) hay ghi (0).

SPI là trao đổi: muốn nhận một byte phải gửi một byte, nên ta gửi 0x00.

CS xuống thấp bắt đầu cuộc nói chuyện; **CS lên cao** kết thúc.

VÍ DỤ

Đọc WHO_AM_I (thanh ghi 0x75): gửi 0xF5 (0x75 | 0x80), rồi 0x00, cảm biến trả lời 0x70 nếu là MPU6500.

CẢM BIẾN

Cài đặt cảm biến trong mpuInit()

Thanh ghi	Giá trị	Ý nghĩa	Vì sao
PWR_MGMT_1	0x80 rồi 0x01	reset, rồi dùng xung PLL của gyro	khởi động sạch; xung ổn định
SMPLRT_DIV	0x03	1 kHz ÷ (1+3) = 250 Hz	mỗi vòng một mẫu mới
CONFIG	0x02	lọc thông thấp 92 Hz	cắt nhiễu rung động cơ
GYRO_CFG	0x08	±500 °/s → 65,5 LSB mỗi °/s	đủ nhanh để lật, vẫn chi tiết
ACCEL_CFG	0x10	±8 g → 4096 LSB mỗi g	dư dải cho va chạm

CẢM BIẾN

Một lần đọc liên tiếp: 14 byte, mọi trục

```
void mpuReadRaw() {
  uint8_t buf[14];
  mpuReadBurst(MPU_REG_ACCEL_XOUT, buf, 14);
  raw_ax = (int16_t)(buf[0]  << 8 | buf[1]);
  raw_ay = (int16_t)(buf[2]  << 8 | buf[3]);
  raw_az = (int16_t)(buf[4]  << 8 | buf[5]);
  /* buf[6..7] = temperature (skip) */
  raw_gx = (int16_t)(buf[8]  << 8 | buf[9]);
  raw_gy = (int16_t)(buf[10] << 8 | buf[11]);
  raw_gz = (int16_t)(buf[12] << 8 | buf[13]);
}
```

Thanh ghi 0x3B…0x48 nằm liền nhau, nên một lần đọc lấy được tất cả.

Vì sao đọc một lượt? Sáu giá trị cùng một thời điểm, và nhanh.

CẢM BIẾN

Hai byte ghép thành một số có dấu

Mỗi giá trị 16 bit, byte cao gửi trước. Ép kiểu int16_t làm nó thành số có dấu.

Số dương

0x01, 0x2C → 0x012C = 300

Số âm

0xFF, 0x38 → 0xFF38 = −200 (bù hai)

Dải

−32768 đến +32767 cho mọi trục

VÍ DỤ

buf[8] = 0xFF, buf[9] = 0x38 → raw_gx = −200 → −200 ÷ 65,5 = −3,05 °/s.

CẢM BIẾN

Từ số đếm thô sang đơn vị thật

65,5

số đếm mỗi °/s của gyro (dải ±500 °/s)

4096

số đếm mỗi g của gia tốc kế (dải ±8 g)

0,004 s

dt: một vòng, dùng để đổi tốc độ thành góc

Ví dụ: gyro thô 655 → 10 °/s. Trong một vòng 4 ms đó là 0,04°.

CẢM BIẾN

Hiệu chuẩn gyro: đo độ lệch khi đứng yên

```
int32_t sx = 0, sy = 0, sz = 0;
for (int i = 0; i < 2000; i++) {
  mpuReadRaw();
  sx += raw_gx; sy += raw_gy; sz += raw_gz;
  delay(3);
}
gyro_x_cal = sx / 2000;    // average = bias
gyro_y_cal = sy / 2000;
gyro_z_cal = sz / 2000;
```

Gyro đứng yên lẽ ra đọc 0, nhưng chip nào cũng lệch một chút — gọi là **độ lệch (bias)**.

2.000 mẫu × 3 ms ≈ 6 giây. Trung bình là độ lệch, được trừ đi mỗi vòng.

Giữ drone phẳng và đứng yên, nếu không độ lệch sẽ sai.

VÍ DỤ

Độ lệch 0,3 °/s không được trừ → sau 60 s hướng sai 0,3 × 60 = 18°.

CẢM BIẾN

Nghiêng từ trọng lực: góc của gia tốc kế

Gia tốc kế tìm độ nghiêng từ phần trọng lực trên mỗi trục.

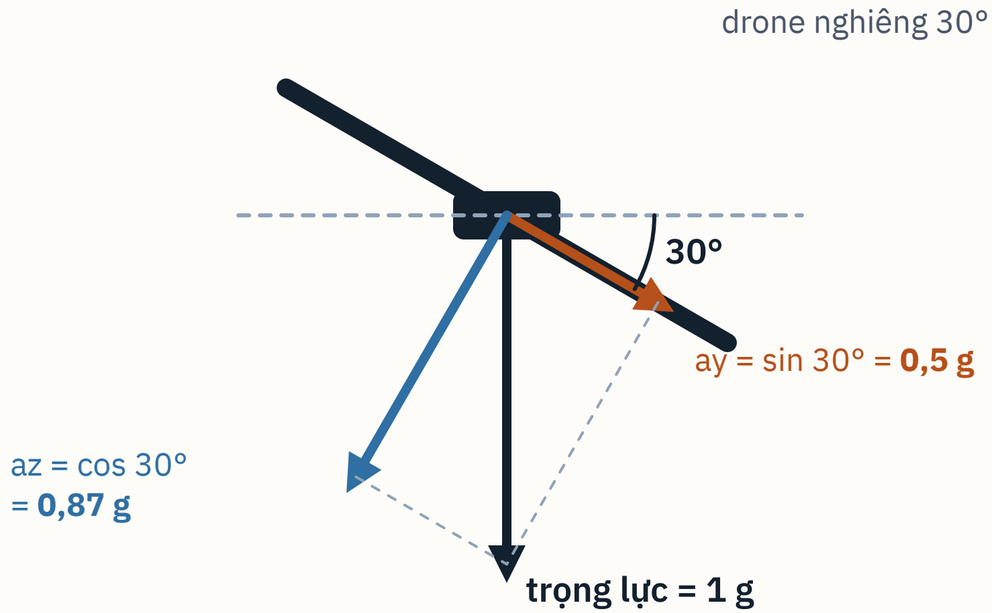
Khi đứng yên chỉ có trọng lực. Nghiêng thì một phần rơi vào trục ngang.

asin(ay ÷ tổng) đổi phần đó ngược lại thành góc.

Chỉ đúng khi drone không tăng tốc: tăng ga hay va chạm làm nó nhảy.

VÍ DỤ

Nghiêng 30°: ay = sin 30° × 1 g = 0,5 g → asin(0,5) = 30°.



CẢM BIẾN

Ý tưởng bộ lọc bù

Kết hợp: tin gyro là chính, và từ từ kéo về theo trọng lực.

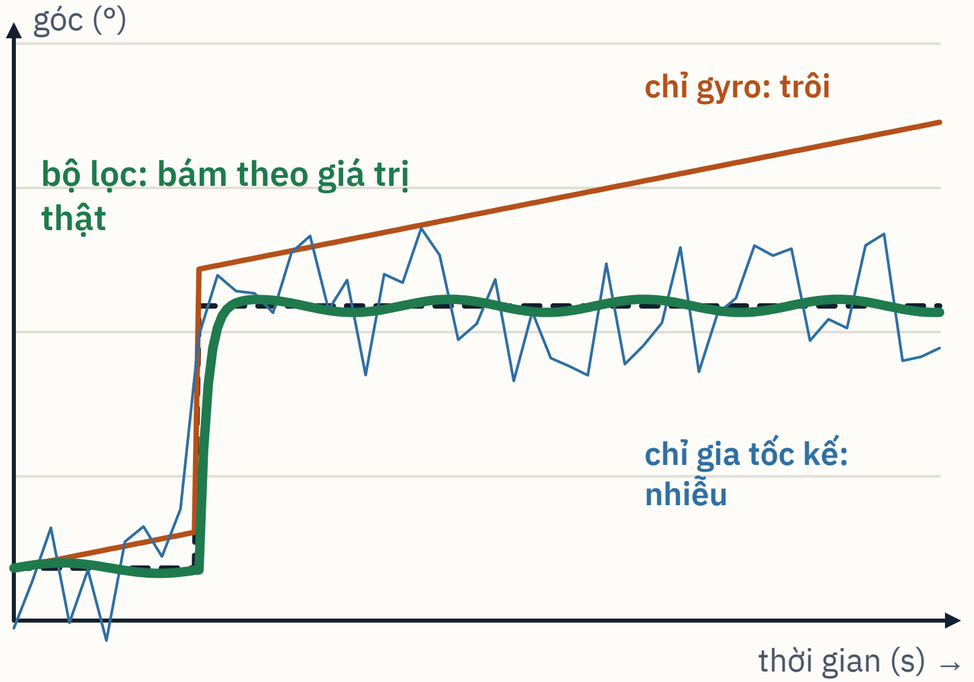
Mỗi vòng: dịch góc theo điều **gyro** báo (nhanh, mượt).

Rồi nhích 0,4% về phía góc **gia tốc kế** (chậm, không trôi).

Kết quả giữ nửa tốt của mỗi cái.

VÍ DỤ

Đường cong minh họa ý tưởng, không phải dữ liệu ghi thật.



Bộ lọc trong mã

```
angle_pitch_deg += gx * dt;      // 1. predict
with gyro
angle_roll_deg  += gy * dt;
angle_pitch_deg = angle_pitch_deg * 0.996f
                + angle_pitch_acc * 0.004f;   //
2. correct
angle_roll_deg  = angle_roll_deg  * 0.996f
                + angle_roll_acc  * 0.004f;
```

Bước 1: dịch góc theo điều gyro nói đã xảy ra trong vòng này.

Bước 2: trộn 99,6% giá trị đó với 0,4% góc gia tốc kế.

Ở vòng đầu tiên, góc bắt đầu từ gia tốc kế (gyro_angles_set).

VÍ DỤ

Góc 10,00°, gyro báo +0,04° vòng này, gia tốc kế báo 9,0° → (10,04 × 0,996) + (9,0 × 0,004) = 10,036°.

CẢM BIẾN

Vì sao 0,996 và 0,004?

0,4%

phần gia tốc kế mỗi vòng: nhỏ, nên rung gần như không lọt vào

≈ 1 s

hằng số thời gian: 0,004 s ÷ 0,004 — tốc độ kéo độ trôi ra

250

vòng mỗi giây, nên các cú nhích 0,4% cộng lại nhanh

Hỏi tại sao: các trọng số này sẽ thế nào nếu vòng lặp chạy ở 500 Hz?

CẢM BIẾN

Hướng bị trôi — và mã nói rõ điều đó

Roll và pitch có trọng lực sửa. Hướng (yaw) không có gì.

Cách tính

heading_deg += gz × dt, quấn trong 0…360°.

Vì sao trôi

Không có la bàn: mọi độ lệch gyro còn sót cộng dồn mãi.

Ảnh hưởng tới

Giữ hướng trong HOLD/AUTO và hệ tọa độ dẫn đường.

VÍ DỤ

Độ lệch còn sót 0,05 °/s → khoảng 3° trôi hướng mỗi phút bay.

CẢM BIẾN · HỎI TẠI SAO

Kiểm tra hiểu biết

Vì sao byte đọc SPI bật bit 7?

Gợi ý: cảm biến phân biệt đọc và ghi thế nào?

Vì sao phải hiệu chuẩn gyro mỗi lần bật nguồn?

Gợi ý: độ lệch có đổi theo nhiệt độ không?

Vì sao không dùng riêng gia tốc kế để đo nghiêng?

Gợi ý: nó cảm nhận gì khi tăng ga đột ngột?

Vì sao roll sửa được mà hướng thì không?

Gợi ý: trọng lực hướng về đâu?

06

Điều khiển PID

Biến “ta sai bao nhiêu?” thành “ta đẩy mạnh bao nhiêu?”

PID qua một ví dụ: vòi sen

P nhìn hiện tại, I nhìn quá khứ, D nhìn tương lai.

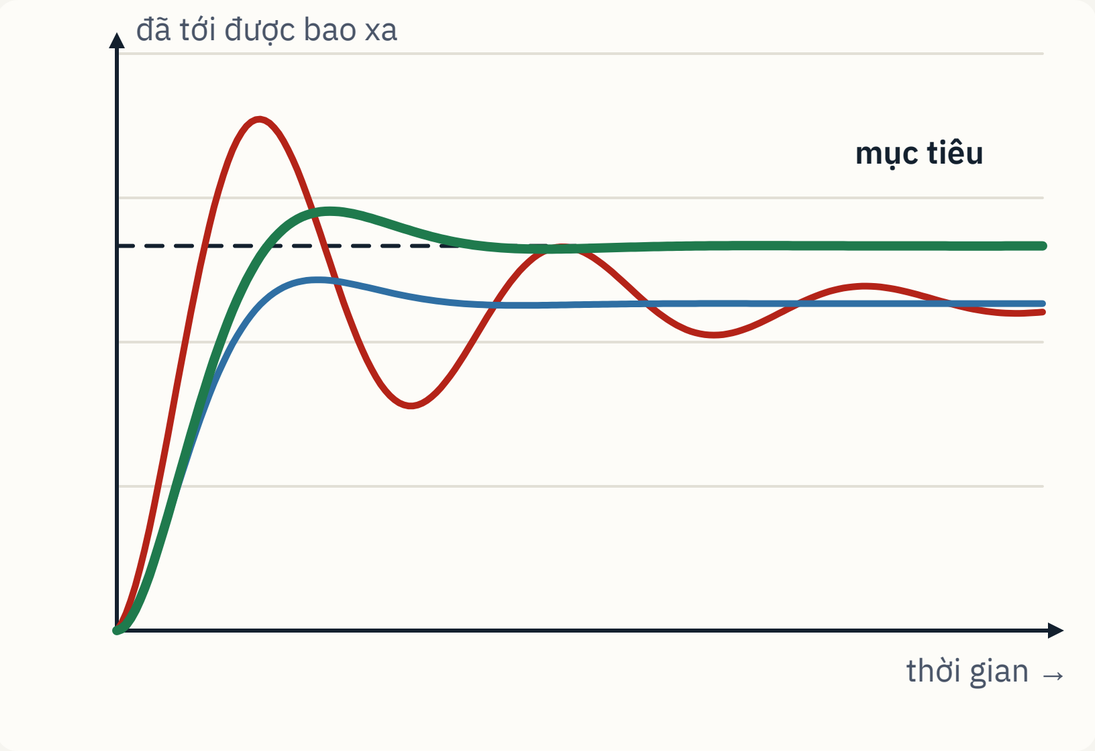
Chỉ P: đẩy mạnh, vượt quá, lắc — và dừng hơi thiếu.

P + D: D phanh dao động nên êm — vẫn hơi thiếu.

P + I + D: I xóa khoảng thiếu cuối, nên dừng đúng mục tiêu.

VÍ DỤ

Vòi sen: P vặn về nóng, I tiếp tục thêm khi còn lạnh, D giảm bớt khi nước nóng lên nhanh.



ĐIỀU KHIỂN PID

P: đẩy tỷ lệ với sai số

P = Kp × sai số. Sai số gấp đôi, lực đẩy gấp đôi.

Kp của roll và pitch

1,3

Kp của yaw

4,0 (mô-men yaw yếu)

P quá lớn

Drone rung nhanh (dao động). Quá nhỏ: cảm giác mềm, lười.

VÍ DỤ

Drone lăn 10 °/s mà ta muốn 0 → sai số 10 → P = 1,3 × 10 = 13 đơn vị sửa động cơ.

ĐIỀU KHIỂN PID

I: sửa sai số không chịu biến mất

I sửa sai số không chịu biến mất.

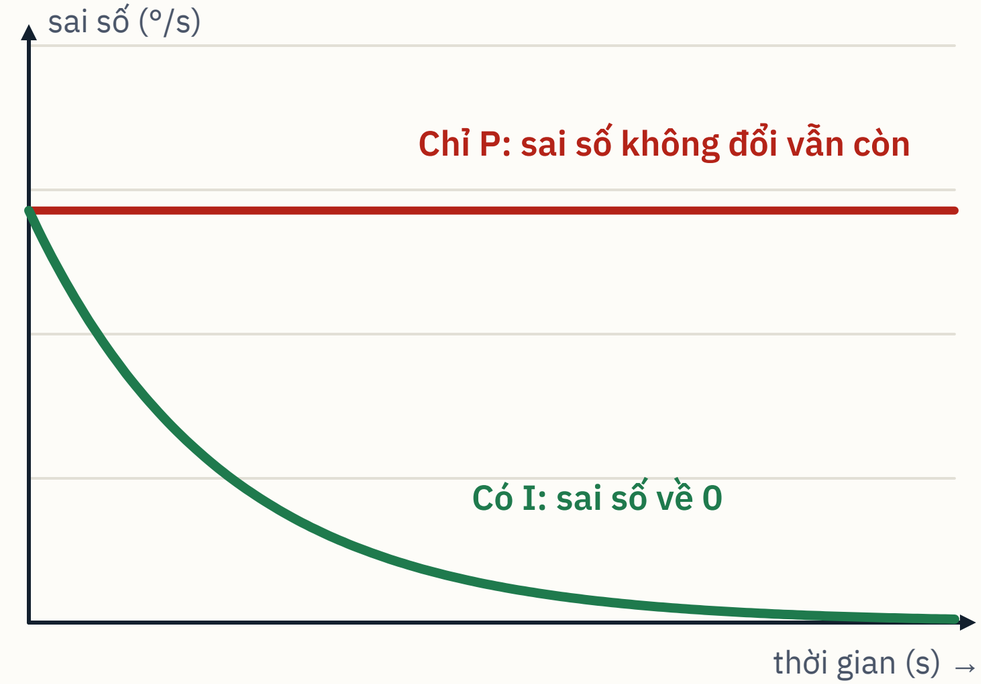
I cộng dồn sai số mỗi vòng: **i_mem += Ki × sai số**.

Sai số nhỏ mà kéo dài thành lệnh sửa lớn.

Ki của roll và pitch = 0,04 — rất nhỏ mỗi vòng, nhưng 250 vòng mỗi giây.

VÍ DỤ

Pin lệch tâm 2 mm → sai số 1 °/s. Sau 1 s: I = 0,04 × 1 × 250 = 10 đơn vị — đủ giữ thăng bằng.



ĐIỀU KHIỂN PID

D: phanh trước khi vượt quá

D phản ứng với tốc độ thay đổi của sai số. Nó là bộ giảm chấn.

Mã

D = Kd × (sai số − sai số trước).

Kd của roll và pitch

18 — lớn, vì thay đổi mỗi vòng 4 ms rất nhỏ.

D quá lớn

Động cơ nóng và rít cao: D khuếch đại nhiễu cảm biến.

VÍ DỤ

Sai số từ 10 xuống 7 trong một vòng → thay đổi −3 → D = 18 × (−3) = −54, phanh lệnh sửa trước khi vượt quá.

pidCompute(): trục roll

```
err = gyro_roll - roll_setpoint;
pid_i_mem_roll += pid_i_gain_roll * err;
pid_i_mem_roll = constrain(pid_i_mem_roll,
                     -I_TERM_LIMIT, I_TERM_LIMIT);
pid_p_roll = pid_p_gain_roll * err;
pid_i_roll = pid_i_mem_roll;
pid_d_roll = pid_d_gain_roll * (err - pid_last_roll_err);
pid_output_roll = pid_p_roll + pid_i_roll + pid_d_roll;
pid_last_roll_err = err;
pid_output_roll = constrain(pid_output_roll,
                     -PID_LIMIT, PID_LIMIT);
```

Pitch và yaw lặp lại mười dòng này với hệ số riêng.

P, I và D được lưu riêng để ứng dụng vẽ biểu đồ từng thành phần (nhóm log **pid**).

ĐIỀU KHIỂN PID

Dấu của sai số rất quan trọng

Ở đây sai số = đo được – mong muốn (gyro – setpoint). Nhiều sách viết mong muốn – đo được.

Cách nào cũng được

...miễn dấu của bộ trộn khớp với lựa chọn.

Quy tắc

Đầu ra dương phải đẩy drone về tốc độ âm.

Nếu một dấu sai

Lệnh sửa thành lệnh đẩy, và drone lật ngay lập tức.

VÍ DỤ

Thử khi tháo cánh: arm, nghiêng drone bằng tay. Động cơ phía thấp phải tăng tốc. Nếu phía cao tăng tốc, có một dấu sai.

Wind-up: vì sao I bị giới hạn và đặt lại

Wind-up: I tăng khi nó không thể tác động.

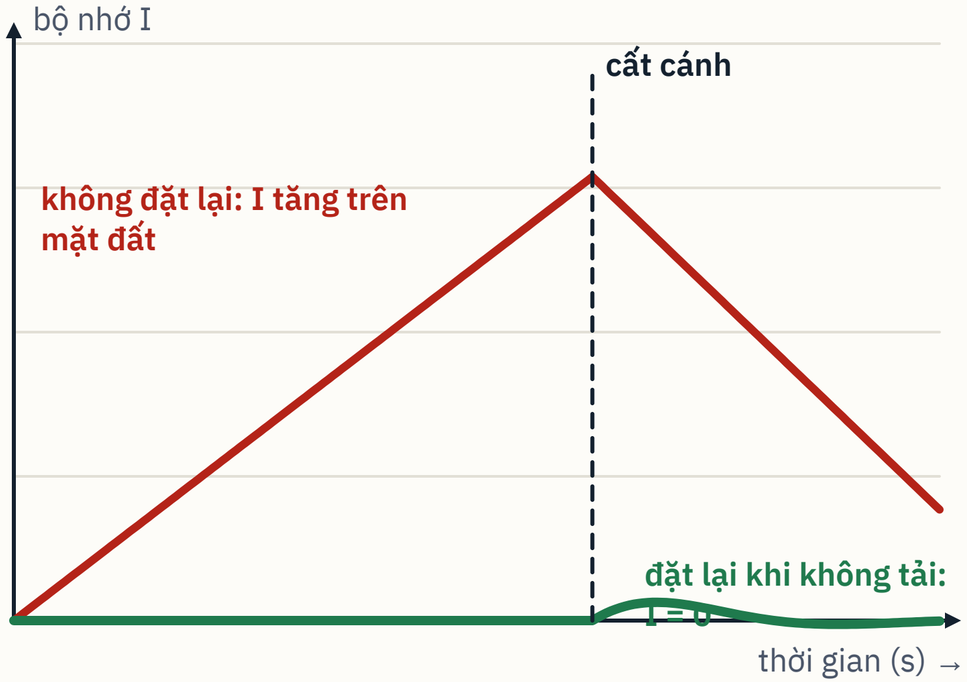
Trên nền gồ ghề, sai số nhỏ không bao giờ hết, nên I cứ tăng.

Lúc cất cánh, I tích trữ đó làm drone nghiêng mạnh.

Hai cách sửa: kẹp I trong ±400, và đặt lại khi ga ở mức không tải.

VÍ DỤ

Sai số 2 trong 5 s trên đất → I = 0,04 × 2 × 1250 = 100 đơn vị tích trữ trước khi cất cánh.



ĐIỀU KHIỂN PID

D theo sai số, và tác dụng phụ của nó

Cả hai giảm chấn chuyển động thật như nhau. Chúng chỉ khác khi setpoint nhảy.

D theo sai số (firmware này)

D = Kd × thay đổi của (gyro – setpoint)

Đơn giản: một công thức

Gạt cần đột ngột đổi setpoint → một xung trong một vòng (derivative kick)

D theo phép đo (cách khác)

D = Kd × thay đổi của riêng gyro

Không có xung khi gạt cần

Mã nhiều hơn chút; lựa chọn phổ biến ở hệ bay lớn

Hỏi tại sao xung ở đây nhỏ: setpoint từ cần thay đổi mượt và đầu ra bị kẹp ±400.

ĐIỀU KHIỂN PID

Các giới hạn giữ cho điều khiển còn khả thi

±400

kẹp đầu ra PID (PID_LIMIT): một trục không chiếm hết dải động cơ

1800

THROTTLE_MAX: ga tối đa còn chừa 200 cho lệnh sửa

1100

mức tối thiểu khi arm mỗi động cơ: cánh không bao giờ dừng trên không

Ví dụ: ga 1800, sửa roll +250 → động cơ phải muốn 2050 → kẹp còn 2000. Không có trần 1800 thì chẳng còn chỗ nào.

ĐIỀU KHIỂN PID

Hai vòng chồng nhau: góc ở ngoài, tốc độ ở trong

Hai vòng chồng nhau: góc ở ngoài, tốc độ ở trong.

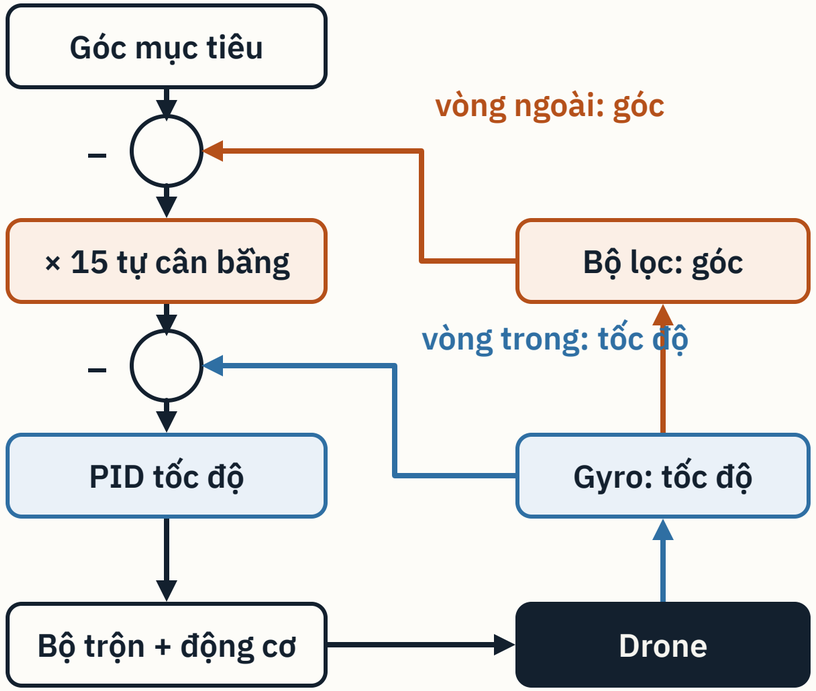
Vòng ngoài: sai số góc × 15 → xoay về nhanh cỡ nào (tốc độ mục tiêu).

Vòng trong: tốc độ mục tiêu – tốc độ gyro → PID đầy đủ → bộ trộn.

Vì sao hai vòng? Vòng tốc độ nhanh chống nhiễu nhanh; vòng góc chỉ quyết định quay về nhanh cỡ nào.

VÍ DỤ

Góc 4°, mục tiêu 0° → vòng ngoài đòi –60 °/s → gyro đọc –20 °/s → sai số trong +40 → đẩy mạnh hơn về.



ĐIỀU KHIỂN PID

Tự cân bằng trong mã

```
float roll_err_deg = angle_roll_deg -
sp_roll_deg;
roll_level_adjust = constrain(
    roll_err_deg * LEVEL_SENSITIVITY, -200.0f,
200.0f);
// The level term is SUBTRACTED.
roll_setpoint = sp_roll_rate - roll_level_adjust;
```

Sai số góc → nhân 15 → tốc độ °/s, kẹp trong ±200.

Bị trừ: nghiêng quá về một phía thì đòi xoay về phía kia.

sp_roll_rate cộng thêm lệnh tốc độ của phi công (SBUS).

VÍ DỤ

Nghiêng +5° quá mục tiêu → adjust = 5 × 15 = 75 → tốc độ mục tiêu −75 °/s → drone xoay về.

ĐIỀU KHIỂN PID

Lỗi thật: dấu của tự cân bằng

Một ký tự, + hay −, quyết định drone có bay được hay không.

Trước (lỗi)

roll_setpoint = sp_roll_rate + roll_level_adjust

Nghiêng +5° → đòi +75 °/s: nghiêng THÊM

Phản hồi dương: drone lật

Sau (đã sửa)

roll_setpoint = sp_roll_rate − roll_level_adjust

Nghiêng +5° → đòi −75 °/s: về thăng bằng

test/level_test.cpp mô phỏng cả hai

level_test giữ công thức cũ làm đối chứng âm, nên bài kiểm thử không thể đỗ một cách tình cờ.

ĐIỀU KHIỂN PID

LEVEL_SENSITIVITY = 15

15

°/s tốc độ quay về cho mỗi độ sai số góc

±200

°/s giới hạn tốc độ tự cân bằng

≈ 13°

sai số tại đó chạm giới hạn (200 ÷ 15)

Ví dụ: sai số 2° → 30 °/s quay về. Sai số 20° → 300 → giới hạn còn 200 °/s.

Từ cần điều khiển tới setpoint

```
// SBUS: rate mode, 16 µs dead-band around centre
if (rc[CH_ROLL] > 1508)
  sp_roll_rate = (rc[CH_ROLL] - 1508) /
SETPOINT_DIVISOR;
else if (rc[CH_ROLL] < 1492)
  sp_roll_rate = (rc[CH_ROLL] - 1492) /
SETPOINT_DIVISOR;
// Phone MANUAL: angles, capped at ±25°
sp_roll_deg = constrain(cmd_roll_deg,
                        -MAX_TILT_DEG,
MAX_TILT_DEG);
```

Vùng chết 1492–1508: cần để gần giữa không ra lệnh gì.

÷ 3 (SETPOINT_DIVISOR): cần hết cỡ 2000 → (2000 – 1508) ÷ 3 = 164 °/s.

Điện thoại gửi **góc**; SBUS gửi **tốc độ**.

ĐIỀU KHIỂN PID

Hệ số mặc định và cách chỉnh

Trục	Kp	Ki	Kd	Dải trên điện thoại (P / I / D)
Roll	1,3	0,04	18	0–100 / 0–20 / 0–200
Pitch	1,3	0,04	18	0–100 / 0–20 / 0–200
Yaw	4,0	0,02	0	0–100 / 0–20 / 0–200

Thứ tự chỉnh: I = 0 → tăng P tới khi rung, giảm 30% → thêm D để hết nảy → thêm I để hết trôi → lưu.

Kiểm tra hiểu biết

Vì sao drone cần I khi P đã sửa?

Gợi ý: P làm gì khi sai số nhỏ nhưng không đổi?

Vì sao I được đặt lại khi ga không tải?

Gợi ý: cái gì tăng lên khi drone đậu trên cỏ?

Vì sao D quá lớn làm động cơ nóng?

Gợi ý: D làm gì với nhiễu cảm biến?

Vì sao thành phần tự cân bằng bị trừ?

Gợi ý: nghiêng +5°, nó phải xoay về phía nào?

07

Bộ trộn và động cơ

Ba lệnh sửa và một mức ga trở thành bốn lệnh động cơ.

Bộ trộn: bốn dòng

```
motor_rf = t - pitch + roll - yaw *
YAW_DIRECTION;
motor_rr = t + pitch + roll + yaw *
YAW_DIRECTION;
motor_lr = t + pitch - roll - yaw *
YAW_DIRECTION;
motor_lf = t - pitch - roll + yaw *
YAW_DIRECTION;
```

t = ga, như nhau cho cả bốn.

Roll: + ở cặp phải, – ở cặp trái.

Pitch: + ở cặp sau, – ở cặp trước.

Yaw: + ở một đường chéo (RR, LF), – ở đường chéo kia (RF, LR).

VÍ DỤ

Dấu mỗi cột cộng lại bằng 0: roll + + – – = 0, pitch – + + – = 0, yaw – + – + = 0.

Ví dụ tính: một vòng trộn

Ví dụ tính: một vòng trộn.

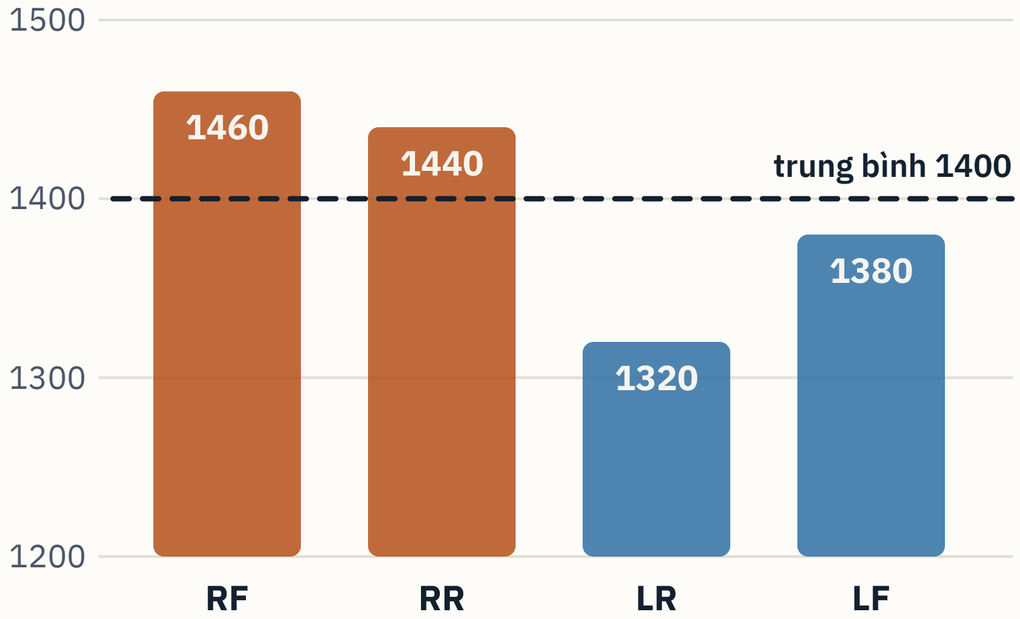
Đầu ra PID vòng này: **pitch −20, roll +50, yaw +10**. Ga 1400.

RF = 1400 + 20 + 50 − 10 = **1460**; RR = 1400 − 20 + 50 + 10 = **1440**.

LR = 1400 − 20 − 50 − 10 = **1320**; LF = 1400 + 20 − 50 + 10 = **1380**.

VÍ DỤ

Trung bình = (1460 + 1440 + 1320 + 1380) ÷ 4 = 1400 — trộn chỉ chuyển lực đẩy qua lại; không thêm chút nào.



Các giới hạn bên trong bộ trộn

```
int16_t t = constrain(throttle, RC_MIN, THROTTLE_MAX);
if (t > THROTTLE_ARM_MAX) {          // above 1050
  ... four mixer lines ...
  motor_rf = constrain(motor_rf, 1100, 2000);
  ...
} else {
  motor_rf = motor_rr = motor_lr = motor_lf = 1000;
  pidReset();   // avoid I-term windup at idle
}
```

Ga bị giới hạn ở **1800** để chừa chỗ cho lệnh sửa.

Mỗi động cơ nằm trong **1100** đến **2000** khi đang bay.

Ở ga không tải (≤ 1050) động cơ dừng và bộ nhớ PID bị xóa.

usToDuty(): từ 1000–2000 sang 0–1023

```
inline uint16_t usToDuty(int16_t us) {
  if (us ⩽ 1000) return 0;
  if (us ⩾ 2000) return MOTOR_PWM_MAX;   // 1023
  return (uint16_t)(((uint32_t)(us - 1000)
                     * MOTOR_PWM_MAX) / 1000);
}
```

Mã nghĩ theo “µs” như ESC của drone lớn, rồi đổi ở bước cuối.

Vì sao giữ 1000–2000? Con số quen thuộc, và cùng bộ trộn vẫn dùng được nếu bạn đổi sang ESC.

Ép kiểu uint32_t để phép nhân không tràn 16 bit.

VÍ DỤ

1500 → (500 × 1023) ÷ 1000 = 511 → khoảng 50% duty. 1100 → 102 → 10% duty, mức tối thiểu khi arm.

Cài đặt phần cứng PWM (LEDC)

```
for (uint8_t i = 0; i < 4; i++) {
  pinMode(pins[i], OUTPUT);
  digitalWrite(pins[i], LOW);
#if ESP_ARDUINO_VERSION_MAJOR ≥ 3
  ledcAttach(pins[i], MOTOR_PWM_FREQ_HZ, MOTOR_PWM_BITS);
#else
  ledcSetup(i, MOTOR_PWM_FREQ_HZ, MOTOR_PWM_BITS);
  ledcAttachPin(pins[i], i);
#endif
}
```

LEDC là khối PWM của ESP32. Nó làm cho LED, nhưng lái MOSFET cũng tốt.

Hai API: core Arduino-ESP32 bản 3.x đổi tên hàm, nên hỗ trợ cả hai.

Các chân được kéo LOW trước: động cơ tắt.

BỘ TRỘN VÀ ĐỘNG CƠ

Vì sao 20 kHz và 10 bit đi cùng nhau

80 MHz

xung nhịp LEDC trên ESP32

4000

nhịp xung mỗi chu kỳ 20 kHz (80 M ÷ 20 k)

1024

bậc duty ở 10 bit — lọt trong 4000; 12 bit (4096) thì không

Độ phân giải × tần số là một đánh đổi. Hỏi tại sao: bạn chọn gì cho động cơ gimbal cần êm?

Mọi chỗ động cơ bị dừng

Ở đâu	Vì sao
setup()	trạng thái an toàn trước khi mọi thứ khởi động
Failsafe trong loop()	không có đầu vào sống trong 500 ms
flightEmergency()	CẮT ĐỘNG CƠ từ ứng dụng — luôn được thực hiện
Cuối hạ cánh AUTO	đường dốc xong → disarm
DISARM, hạ ga, gạt cần disarm	phi công yêu cầu

BỘ TRỘN VÀ ĐỘNG CƠ

Bão hòa: khi một động cơ chạm giới hạn

Nếu một động cơ cần hơn 2000 hoặc dưới 1100, nó bị cắt. Cân bằng giữa bốn động cơ mất đi.

Bạn thấy gì

Ở ga tối đa drone kém ổn định hơn; có thể xoay hoặc lắc.

Vì sao

Động cơ bị cắt không đóng góp được phần sửa của nó.

Hỗ trợ có sẵn

THROTTLE_MAX 1800 và PID_LIMIT ±400 giảm khả năng bị cắt.

VÍ DỤ

Ga 1800, roll +300 → động cơ phải 2100 → cắt còn 2000. Mất 100 đơn vị sửa roll.

Kiểm tra hiểu biết

Vì sao các dấu của bộ trộn ở mỗi cột cộng lại bằng 0?

Gợi ý: tổng lực đẩy thay đổi thế nào?

Vì sao giữ 1000–2000 khi động cơ dùng duty 0–1023?

Gợi ý: nghĩ về ESC và tính dễ đọc.

Vì sao ga bị giới hạn ở 1800?

Gợi ý: lệnh sửa cần gì ở ga tối đa?

Vì sao PWM 10 bit mà không 12 bit ở 20 kHz?

Gợi ý: đếm số nhịp xung mỗi chu kỳ.

08

Đầu vào vô tuyến: SBUS và CRTP

Cần điều khiển và cú chạm trên điện thoại trở thành con số trong vòng lặp ra sao.

ĐẦU VÀO

Một khung SBUS: 25 byte

Một khung SBUS: 25 byte, mỗi 7–14 ms.

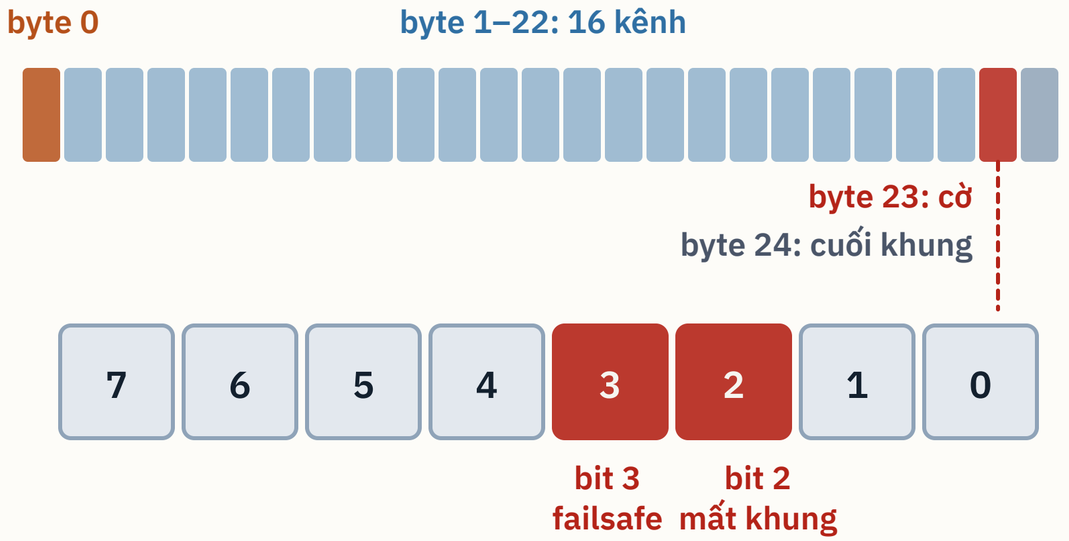
Byte 0: đầu khung 0x0F, bắt đầu một khung.

Byte 1–22: 16 kênh × 11 bit, gói sát nhau.

Byte 23: cờ. **Byte 24**: cuối khung (0x00, hoặc 0x04/0x14/0x24/0x34 với SBUS2).

VÍ DỤ

Lỗi cũ đọc cờ từ byte 24 — byte cuối khung — nên máy thu ở failsafe không bao giờ bị phát hiện.



Một trong hai bit bật → khung bị bỏ, và hạn 500 ms sẽ dừng động cơ.

ĐẦU VÀO

Tách các kênh 11 bit

```
sbus_raw[0] = (b[1]      | b[2] << 8)
& 0x7FF;
sbus_raw[1] = (b[2] >> 3 | b[3] << 5)
& 0x7FF;
sbus_raw[2] = (b[3] >> 6 | b[4] << 2 | b[5] <<
10) & 0x7FF;
...    // 13 more lines, same pattern
```

Các kênh không thẳng hàng với byte: mỗi kênh 11 bit nên vắt qua ranh giới byte.

Dịch bit đưa bit vào đúng chỗ; **OR** ghép các mảnh; **& 0x7FF** giữ 11 bit.

Giá trị 0–2047; cần bình thường dùng khoảng 172–1811.

VÍ DỤ

b[1] = 0xE0, b[2] = 0x03 → (0xE0 | 0x03 << 8) & 0x7FF = 0x3E0 = 992, vị trí giữa của cần.

ĐẦU VÀO

Lỗi thật: đọc nhầm byte

Lệch một byte, cờ failsafe không bao giờ được thấy.

Trước (lỗi)

Đọc cờ từ byte 24, tức byte cuối khung

Byte cuối luôn là 0x00 → cờ luôn trông như sạch

Máy thu ở failsafe vẫn giữ động cơ chạy theo cần cũ

Sau (đã sửa)

Đọc cờ từ byte 23

Khung mất hoặc failsafe bị bỏ

Thời hạn 500 ms sau đó dừng động cơ; sbus_test.cpp ghim lại

ĐẦU VÀO

Tìm khung và co giãn giá trị cần

```
while (Serial2.available()) {
  uint8_t c = Serial2.read();
  if (sbus_idx == 0 && c != 0x0F) continue;  // wait for
header
  sbus_buf[sbus_idx++] = c;
  if (sbus_idx >= SBUS_FRAME_LEN) {
    sbus_idx = 0;
    if (sbusFrameUsable(sbus_buf)) sbusDecode(sbus_buf);
  }
}
// in sbusDecode: 172..1811 -> 1000..2000
rc[i] = constrain(map(sbus_raw[i], 172, 1811, 1000,
2000),
                  1000, 2000);
```

Không chặn: chỉ đọc những gì đã tới, không bao giờ chờ.

map() co giãn về dải 1000–2000 mà phần còn lại của mã dùng.

CH5 thành công tắc đơn giản: trên 1024 → 2000, ngược lại 1000.

ĐẦU VÀO

Arm bằng tay điều khiển RC

Thao tác	Ga	Cần yaw hoặc CH5	Giữ
Arm	dưới 1050	yaw phải (trên 1700) hoặc CH5 cao	1 s
Disarm	dưới 1050	yaw trái (dưới 1300)	1 s
Disarm (công tắc)	bất kỳ	CH5 hạ thấp, khi đã thấy dùng công tắc	1 s

Mọi cách arm đều cần pin từ 3,3 V trở lên (armChecksOk).

ĐẦU VÀO

CRTP: ngôn ngữ của điện thoại

CRTP: byte đầu tiên cho biết gói đi đâu.

CRTP đến từ dự án Crazyflie; ứng dụng ESP-Drone nói giao thức này.

4 bit cao của byte đầu chọn **port**, 2 bit thấp chọn **kênh**.

crtpDispatch() đọc port và gọi đúng handler.

VÍ DỤ

port = (byte >> 4) & 0x0F; kênh = byte & 0x03. Với 0x71: port 7, kênh 1.



bit 7–4: **port** (dịch vụ) 3–2: link 1–0: **kênh**

0x30	0011 0000	port 3, kênh 0 → commander (cần lái)
0x70	0111 0000	port 7, kênh 0 → điều khiển bay
0x71	0111 0001	port 7, kênh 1 → dẫn đường
0x25	0010 0101	port 2, kênh 1 → đọc tham số

ĐẦU VÀO

Đóng khung và checksum

```
uint8_t crtpChecksum(const uint8_t *data, size_t len) {
  uint32_t sum = 0;
  for (size_t i = 0; i < len; i++) sum += data[i];
  return (uint8_t)(sum & 0xFF);
}
// on receive (UDP / SPP):
if (crtpChecksum(data, len - 1) ≠ data[len - 1]) return;
crtpDispatch(data, len - 1);
crtp_last_rx_ms = millis();     // the link is alive
```

Qua UDP và SPP, mỗi gói kết thúc bằng một byte checksum: 8 bit thấp của tổng.

Checksum sai → gói bị bỏ im lặng.

Gói tốt cũng cập nhật **crtp_last_rx_ms**, được failsafe dùng.

VÍ DỤ

Gói 0x70 0x02 → tổng 0x72 → byte checksum 0x72. Trên đường truyền: 70 02 72.

ĐẦU VÀO

Ba cách mang CRTP

Đường truyền	Chi tiết	Đóng khung
Wi-Fi UDP	drone 192.168.43.42, nghe cổng 2390, trả lời về 2399	CRTP + checksum
Bluetooth SPP	nối tiếp qua Bluetooth Classic (Android)	CRTP + checksum
Bluetooth LE	ghi và thông báo GATT (iPhone)	đoạn 19 byte, không checksum

ĐẦU VÀO

Gói commander (port 3)

Byte	Trường	Kiểu	Ý nghĩa
0	đầu gói	0x30	port 3, kênh 0
1 – 4	roll	float	góc nghiêng, độ
5 – 8	pitch	float	góc nghiêng, độ
9 – 12	yaw	float	tốc độ yaw, °/s
13 – 14	thrust	uint16	0 – 65535

Kiểm tra hợp lệ: NaN hoặc vô cực → bỏ qua; |roll| hoặc |pitch| quá 180, hay |yaw| quá 2000 → bỏ qua.

ĐẦU VÀO

Các port CRTP được dùng

Port	Tên	Xử lý?	Dùng cho
0	console	chỉ gửi	tin nhắn chữ tới ứng dụng (pin yếu, từ chối arm)
2	params	có	đọc và ghi giá trị chỉnh được
3	commander	có	gói cần điều khiển ở MANUAL
5	logging	có	truyền telemetry, tối đa 3 khối
7	generic	có	kênh 0 điều khiển bay, kênh 1 dẫn đường
4, 15	memory, link	bỏ qua	bo mạch này không cần

ĐẦU VÀO

Tham số: TOC và kiểm tra dải

```
{ "pid", "roll_kp", PT_FLOAT, 4, TOC_ACCESS_RW,
  &pid_p_gain_roll, 0.0f, 100.0f },
{ "flight", "takeoff_ms", PT_UINT16, 2,
TOC_ACCESS_RW,
  &takeoff_ms, 100.0f, 20000.0f },
// on write:
if (!isfinite(v)) return;     // refused
v = paramClampF(e, v);        // clamp into [lo,
hi]
```

TOC = mục lục: ứng dụng tải danh sách tên, kiểu và quyền truy cập.

Mỗi mục đọc-ghi mang dải **lo** và **hi**.

Phản hồi lặp lại giá trị thật đã lưu, nên ứng dụng hiện đúng sự thật.

VÍ DỤ

Ứng dụng ghi roll_kp = 1000 → lưu thành 100 (giới hạn hi) → phản hồi báo ứng dụng 100.

ĐẦU VÀO

Log: truyền dữ liệu về điện thoại

Ứng dụng chọn biến từ TOC log và đặt chu kỳ. Sau đó drone tự gửi.

Tối đa 3 khối

Mỗi khối tới 12 biến và chu kỳ riêng (bước 10 ms).

Các nhóm

pm.vbat, stabilizer, controller, motor, flight, nav, pid (P/I/D mỗi trục).

Gói

id khối, mốc thời gian 24 bit, rồi giá trị lấy thẳng từ biến đang chạy.

VÍ DỤ

Khối 1, chu kỳ 25 (× 10 ms = 250 ms): pm.vbat, flight.mode, flight.armed → 4 gói mỗi giây.

ĐẦU VÀO

Port 7: các lệnh điều khiển bay

Mã	Lệnh	Làm gì
0x01	SET_MODE	MANUAL, HOLD, AUTO, FOLLOW, RTH (từ chối khi pin nguy hiểm)
0x02 / 0x03	TAKEOFF / LAND	bắt đầu dốc lên / dốc xuống
0x04	SET_HOVER	lực đẩy lơ lửng, kẹp trong 8000 – 55000
0x05	EMERGENCY	tắt động cơ ngay — luôn được thực hiện
0x07 / 0x08	ARM / DISARM	ARM chỉ từ trạng thái nghỉ; DISARM luôn được
0x09 – 0x0B	SAVE / DEFAULTS / LOAD	cài đặt trong flash; SAVE bị từ chối khi arm
0x10	LINK	đổi cấu hình kết nối và khởi động lại; từ chối khi arm

ĐẦU VÀO · HỎI TẠI SAO

Kiểm tra hiểu biết

Vì sao phải kiểm tra cờ failsafe của SBUS?

Gợi ý: máy thu gửi gì sau khi mất tay phát?

Vì sao đầu gói CRTP đặt port ở các bit cao?

Gợi ý: crtpDispatch tìm handler thế nào?

Vì sao lặp lại giá trị đã lưu sau khi ghi tham số?

Gợi ý: nếu drone đã kẹp giá trị thì sao?

Vì sao mọi lệnh port 7 phải idempotent?

Gợi ý: ứng dụng làm gì mỗi 500 ms?

09

Chế độ bay và an toàn

Drone tự làm gì, và mọi lớp bảo vệ ngăn nó gây hại.

Năm chế độ bay

Chế độ	Ai lái	Drone giữ gì
MANUAL	phi công (cần điện thoại hoặc RC)	không gì tự động — cần ra lệnh góc hoặc tốc độ
HOLD	drone	thăng bằng, khóa hướng, một mức lực đẩy cố định
AUTO	drone	dốc cất cánh → lơ lửng → dốc hạ cánh
FOLLOW	drone + GPS điện thoại	nghiêng về vị trí mục tiêu điện thoại gửi
RTH	drone	nghiêng quay về điểm cất cánh

Các chế độ chỉ áp dụng cho đường điện thoại. SBUS luôn bay MANUAL.

AUTO là một máy trạng thái nhỏ

AUTO là máy trạng thái nhỏ: mỗi trạng thái một quy tắc đơn giản.

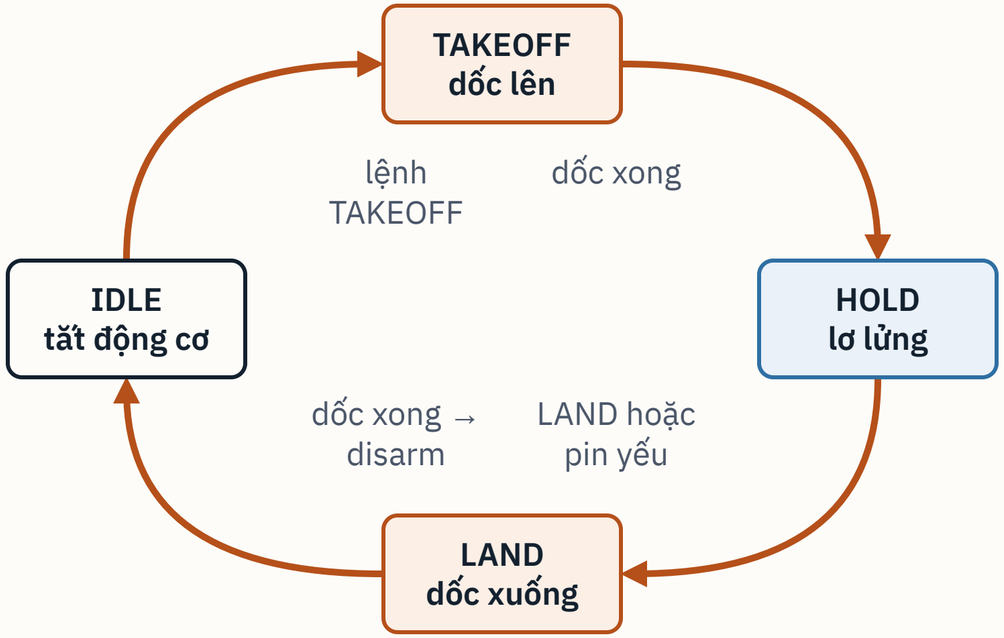
TAKEOFF tự chuyển sang HOLD khi dốc xong.

HOLD kéo dài tới khi chạm LAND hoặc pin vào mức nguy hiểm.

LAND kết thúc bằng disarm và về MANUAL.

VÍ DỤ

Chạm Cất cánh → lên 2 s → lơ lửng → chạm Hạ cánh → xuống 2,5 s → disarm.



CHẾ ĐỘ VÀ AN TOÀN

Các đường dốc cất và hạ cánh

Cất cánh và hạ cánh là các đường dốc thẳng.

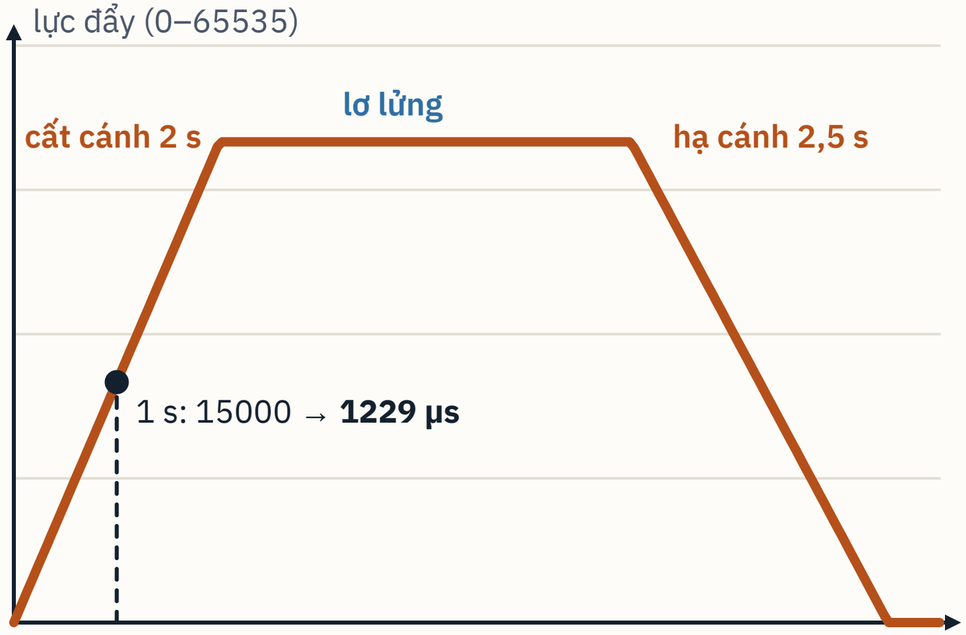
t chạy từ 0 đến 1 trong mỗi pha, nên lực đẩy là đường thẳng.

Lực đẩy (0–65535) đổi sang 1000–2000: **1000 + lực đẩy ÷ 65535 × 1000**.

Hạ cánh hoàn toàn theo thời gian — không có cảm biến nào thấy mặt đất.

VÍ DỤ

hover_thrust 30000, takeoff_ms 2000 → tại 1 s lực đẩy là 15000 → 1229 µs.



CHẾ ĐỘ VÀ AN TOÀN

Giữ hướng trong HOLD và AUTO

Giữ hướng: luôn xoay theo đường ngắn.

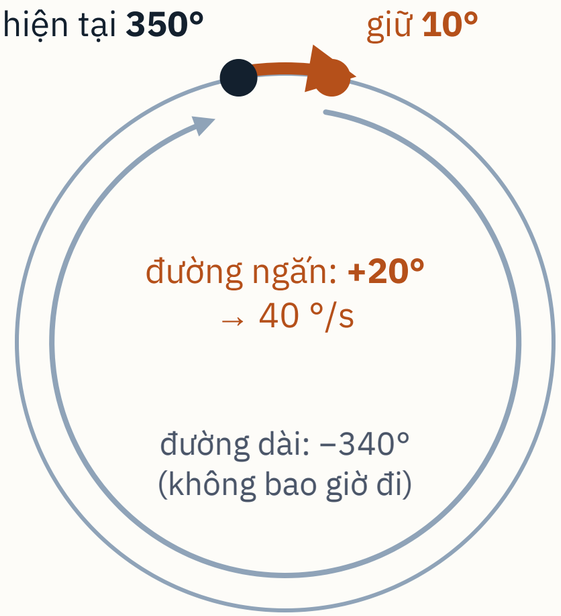
Sai số hướng → tốc độ yaw: **2 °/s mỗi độ**, giới hạn 200 °/s.

Phép quấn (±360 khi vượt ±180) chọn đường ngắn.

Hướng cần giữ được ghi lại khi vào HOLD hoặc lúc cất cánh.

VÍ DỤ

Giữ 10°, hướng 350° → herr = −340 → +360 = +20 → tốc độ yaw 40 °/s.



CHẾ ĐỘ VÀ AN TOÀN

Arm từ điện thoại: ba quy tắc

Ga thấp trước

Ở MANUAL, tăng ga chỉ arm sau khi ứng dụng đã báo ga bằng 0. Điện thoại kết nối khi cần đang cao không thể làm cánh quay.

ARM chỉ từ trạng thái nghỉ

ARM bị từ chối ở HOLD, FOLLOW hay RTH, nơi động cơ sẽ nhảy lên lực đẩy hover. Ở đó hãy dùng TAKEOFF.

TAKEOFF arm an toàn

Nó kiểm tra pin, đặt điểm cất cánh và bắt đầu dốc từ 0.

Một kiểm tra trước arm cho mọi đường

```
bool armChecksOk(const char *who) {
#if VBAT_ARM_CHECK
  if (vbat < VBAT_CELLS * VBAT_CELL_CRIT) {      // 3.3 V
    // rate-limited message, at most once a second
    Serial.printf("[ARM] %s refused: battery %.2f V\n",
who, vbat);
    crtpConsole("arm refused: low battery");
    return false;
  }
#endif
  return true;
}
```

Được gọi bởi arm SBUS, tăng ga CRTP, ARM và TAKEOFF.

Vì sao một hàm? Thêm một kiểm tra ở đây là bảo vệ mọi đường cùng lúc.

Tin nhắn bị giới hạn tần suất để cần giữ lâu không làm ngập console.

CHẾ ĐỘ VÀ AN TOÀN

Failsafe: không có đầu vào, không có động cơ

Failsafe: không đầu vào, không động cơ.

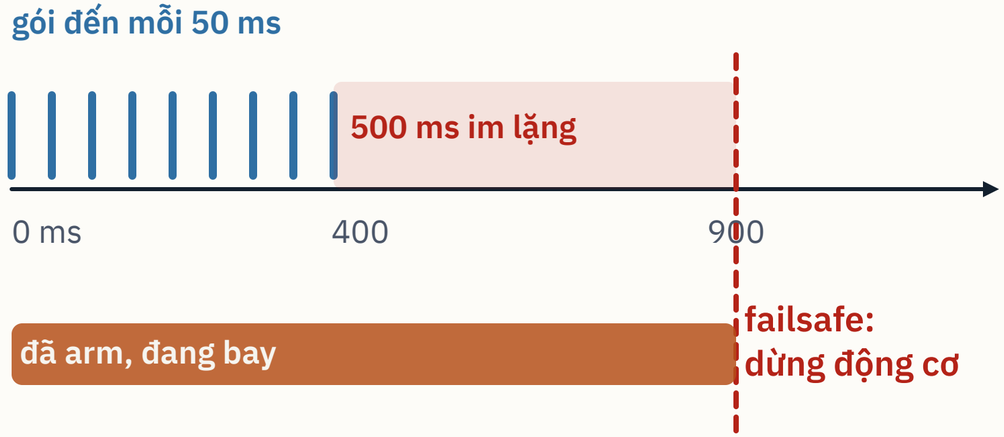
Mỗi đầu vào “còn sống” nếu được nghe trong **500 ms** vừa qua.

rx_ok = điện thoại sống HOẶC SBUS sống.

Không có đầu vào sống khi đang arm → disarm, tắt động cơ, về MANUAL.

VÍ DỤ

Ứng dụng treo ở 400 ms → ở 900 ms cánh dừng, ngay trong vòng mà kiểm tra thất bại.



MANUAL cần gói cần lái. HOLD / AUTO / FOLLOW / RTH chấp nhận mọi lưu lượng CRTP.

CHẾ ĐỘ VÀ AN TOÀN

Lỗi thật: thời gian bị đo ngược

Một mili giây, đọc sai thứ tự, có thể cắt động cơ.

Trước (lỗi)

So sánh tuổi gói bằng số không dấu

Gói xử lý trong vòng này có thể mới hơn *now* 1 ms

now – last = –1 → quấn thành ~49 ngày → failsafe → cắt động cơ

Sau (đã sửa)

(int32_t)(now – last) < FAILSAFE_MS

–1 ms đơn giản là “rất mới”

So sánh có dấu vẫn đúng qua lần quấn 49 ngày

Ai đang điều khiển? Vài tình huống

Tình huống	phone_ok	sbus_ok	Kết quả
Điện thoại bay MANUAL, đang gửi cần	có	—	điện thoại lái
Điện thoại ở AUTO lơ lửng, chỉ có log	có	—	điện thoại lái (AUTO)
Ứng dụng đóng, có máy thu RC	không	có	RC lái, rate thủ công
Mất điện thoại, không có máy thu	không	không	failsafe: dừng động cơ
Tay phát RC tắt, máy thu ở failsafe	không	không (cờ)	failsafe: dừng động cơ

Pin nguy hiểm phải kéo dài 3 giây

Pin nguy hiểm phải kéo dài 3 giây.

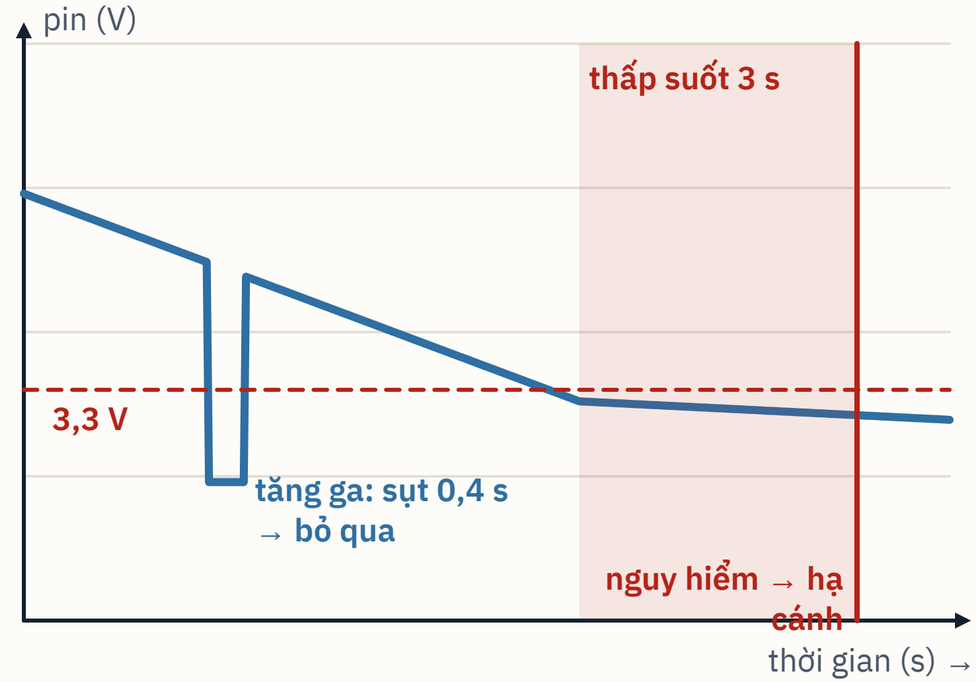
Hai bộ lọc: làm mượt 0,2 s, rồi **giữ 3 s**.

Tăng ga đột ngột làm sụt áp chốc lát — đó không phải pin hết.

Đồng hồ đếm lại ngay khi điện áp hồi trên 3,3 V.

VÍ DỤ

Lơ lửng ở 3,28 V trong 3 s → nguy hiểm → ở AUTO drone tự hạ cánh; phi công lái tay nhận cảnh báo.



CHẾ ĐỘ VÀ AN TOÀN

Pin nguy hiểm → drone tự hạ cánh

```
if (armed && vbat_crit) {
  if (use_crtp && flight_mode ≠ MODE_MANUAL) {
    if (!(flight_mode == MODE_AUTO && auto_state ==
AUTO_LAND))
      flightLand();                    // land once
  } else {
    // RC or MANUAL pilot: warn every 2 s, do not take
over
    crtpConsole("battery critical: land now");
  }
}
// and SET_MODE is refused while armed && vbat_crit
```

Chế độ tự động thì **hạ cánh**; phi công người thì nhận **cảnh báo**.

SET_MODE bị từ chối để việc ứng dụng gửi lại chế độ định kỳ không hủy việc hạ cánh.

CHẾ ĐỘ VÀ AN TOÀN

Số xấu không bao giờ tới động cơ

Mọi giá trị từ bên ngoài đều được kiểm tra trước khi dùng.

Gói commander

NaN, vô cực hoặc góc vô lý → bỏ qua.

Ghi tham số

NaN bị từ chối; còn lại kẹp vào dải TOC.

Lực đẩy hover

kẹp trong 8000 – 55000 từ SET_HOVER, TAKEOFF và flash.

Flash lúc khởi động

hệ số đã lưu không hữu hạn → khôi phục toàn bộ mặc định.

CHẾ ĐỘ VÀ AN TOÀN

Những gì bị từ chối khi đang arm

SAVE

Xóa flash làm cả hai nhân CPU đứng vài mili giây — lỡ một vòng điều khiển.

LINK

Ghi flash và khởi động lại. Khởi động lại trên không là rơi.

Serial link / sta / save

Console trên bàn có cùng quy tắc: “refused while armed — disarm first”.

VÍ DỤ

Chạm Save ở màn hình Tune khi đang lơ lửng → trạng thái 1 → ứng dụng báo việc lưu bị từ chối.

CHẾ ĐỘ VÀ AN TOÀN

CẮT ĐỘNG CƠ, từ đầu đến cuối

Chạm
2 lần ở Fly, 1 lần ở Mission và Tune

→

Ứng dụng gửi
port 7: 0x70 0x05 (EMERGENCY)

→

Firmware
flightEmergency(): disarm, motorStop, pidReset

→

Chế độ
về MANUAL, AUTO nghỉ

→

Phản hồi
0x70 0x05 0x00 → ứng dụng báo đã dừng

EMERGENCY không có điều kiện. Đó là lệnh duy nhất luôn có tác dụng.

VÍ DỤ

Drone ở HOLD trôi về phía tường → chạm hai lần CẮT ĐỘNG CƠ → cánh dừng trong một vòng 4 ms sau khi gói tới.

CHẾ ĐỘ VÀ AN TOÀN

Vì sao TAKEOFF lặp lại bị bỏ qua

```
// A retransmitted TAKEOFF must not restart the
ramp
if (armed && flight_mode == MODE_AUTO &&
    (auto_state == AUTO_TAKEOFF || auto_state ==
AUTO_HOLD))
  return true;                          // already
done
// Flying in another mode: never ramp from zero
bool spinning = motor_rf > 1000 || motor_rr >
1000 || ...;
if (armed && spinning) return false;
```

Ứng dụng gửi lại TAKEOFF mỗi 500 ms tới khi thấy xác nhận.

Nếu mất xác nhận, TAKEOFF thứ hai đến khi đang bay lên.

Bắt đầu lại dốc sẽ đưa lực đẩy về 0 — và làm rơi drone.

An toàn theo lớp

Không kiểm tra nào hoàn hảo, nên chúng được xếp chồng.

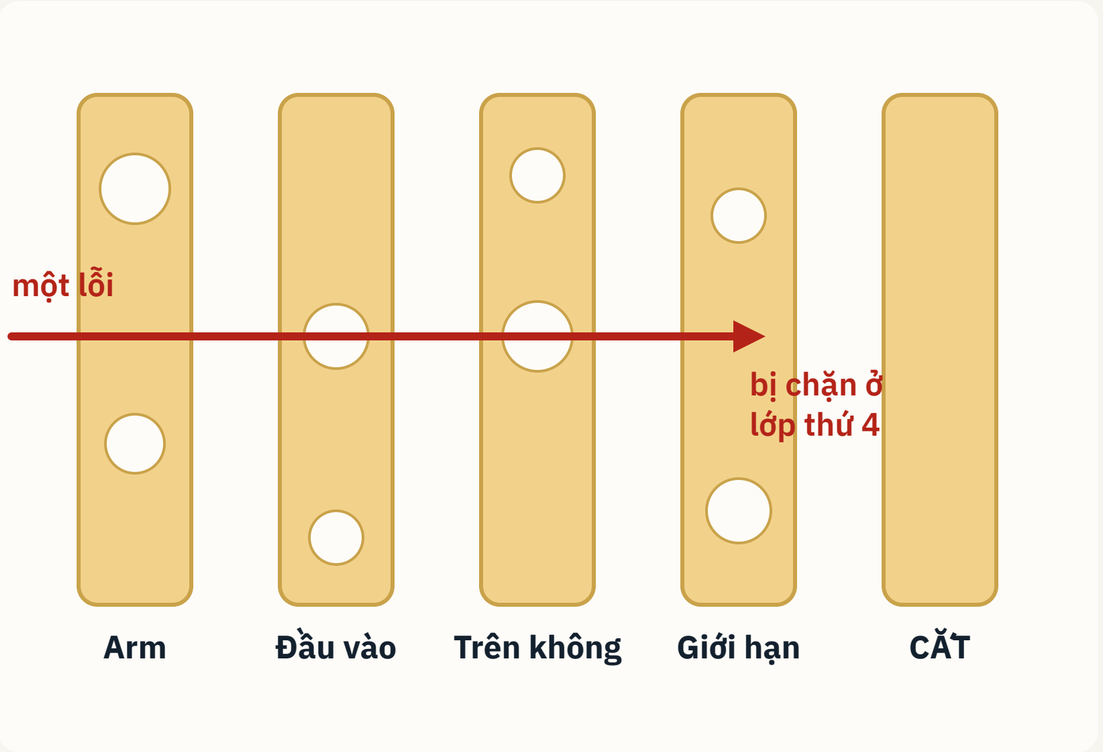
Arm: pin ≥ 3,3 V, thấy ga thấp, ARM chỉ từ nghỉ.

Đầu vào: checksum, kiểm tra NaN và dải, cờ SBUS.

Trên không: failsafe 500 ms, thời gian có dấu, tự hạ cánh.

Giới hạn: nghiêng ±25°, PID ±400, ga 1800, sàn động cơ 1100.

CẮT ĐỘNG CƠ: thực hiện ở mọi trạng thái.



Kiểm tra hiểu biết

Vì sao MANUAL cần gói cần mà AUTO chấp nhận mọi lưu lượng?

Gợi ý: setpoint ở mỗi chế độ là gì?

Vì sao so sánh tuổi thời gian bằng số có dấu?

Gợi ý: 0 – 1 là bao nhiêu với số 32 bit không dấu?

Vì sao tự hạ cánh ở AUTO nhưng chỉ cảnh báo ở MANUAL?

Gợi ý: ai đang lái trong mỗi trường hợp?

Vì sao EMERGENCY là lệnh duy nhất không có điều kiện?

Gợi ý: bao giờ bạn muốn nó bị từ chối?

10

Dẫn đường

Bay về phía điện thoại khi trên drone không có GPS — và trung thực về độ trôi.

DẪN ĐƯỜNG

Ước tính vị trí (dead reckoning): đoán vị trí từ chuyển động

Điện thoại biết nó ở đâu (GPS). Drone chỉ biết nó đã tăng tốc thế nào.

Điện thoại

Tính mục tiêu theo số mét về phía bắc và phía đông so với điểm cất cánh, rồi gửi đi.

Drone

Cộng dồn gia tốc của chính nó để đoán đã đi bao xa từ lúc cất cánh.

Bộ điều khiển

Nghiêng về phía chênh lệch giữa mục tiêu và vị trí đoán.

VÍ DỤ

Đi qua căn phòng tối bằng cách đếm bước: ổn cho vài mét, lệch nhiều sau quãng dài.

DẪN ĐƯỜNG

Gia tốc → vận tốc → vị trí

Gia tốc → vận tốc → vị trí, và độ trôi đi kèm.

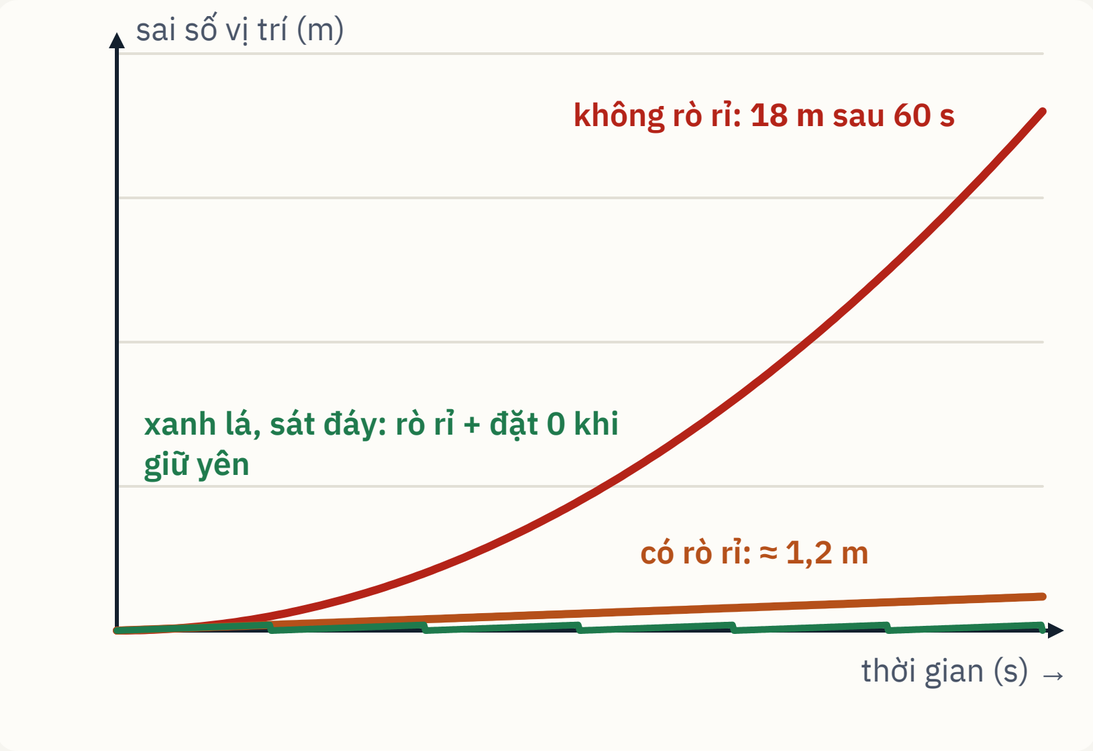
Gia tốc kế → bỏ trọng lực → xoay → tích phân ra vận tốc → tích phân ra vị trí.

Mỗi lần tích phân cộng dồn sai số nhỏ: độ lệch nhỏ tăng theo **bình phương thời gian**.

Ba cách sửa: rò rỉ vận tốc, đặt vận tốc 0 khi giữ yên, đặt lại khi cất cánh.

VÍ DỤ

Độ lệch 0,01 m/s² → ½ × 0,01 × 60² = 18 m sau một phút nếu không rò rỉ. (Khoảng giữ yên là minh họa.)



DẪN ĐƯỜNG

Bỏ trọng lực

```
const float ax_b = raw_ax / NAV_ACC_LSB_PER_G *
G;   // m/s²
const float ay_b = raw_ay / NAV_ACC_LSB_PER_G *
G;
// gravity seen by the tilted sensor
const float gx_g = -G * sin(roll) * cos(pitch);
const float gy_g = +G * sin(pitch);
float fwd  = (ay_b - gy_g) * NAV_FWD_SIGN;
float left = (ax_b - gx_g) * NAV_LEFT_SIGN;
```

Drone nghiêng cảm nhận một phần trọng lực trên các trục ngang, kể cả khi đứng yên.

Góc nghiêng từ phần 5 cho biết bao nhiêu; trừ nó đi.

Phần còn lại là gia tốc ngang thật.

VÍ DỤ

Chúc 5° về trước, không di chuyển: ay_b ≈ 9,81 × sin 5° = 0,85 m/s². Trừ gy_g = 0,85 → fwd = 0.

DẪN ĐƯỜNG

Lỗi thật: hệ số tỷ lệ gia tốc kế

Hai con số, cách nhau 1.800 dòng, phải khớp nhau. Chúng đã không khớp.

Trước (lỗi)

NAV_ACC_LSB_PER_G = 16384 (thang ±2 g)

Cảm biến thực ra đặt ±8 g (4096 mỗi g)

Mọi gia tốc đọc nhỏ đi 4 lần → vị trí nhỏ đi 4 lần

Sau (đã sửa)

NAV_ACC_LSB_PER_G = 4096

Chú thích: phải khớp MPU_REG_ACCEL_CFG trong mpuInit()

run.sh đọc cả hai giá trị và báo lỗi nếu lệch

DẪN ĐƯỜNG

Bộ tích phân có rò rỉ

Bộ tích phân rò rỉ quên dần vận tốc cũ.

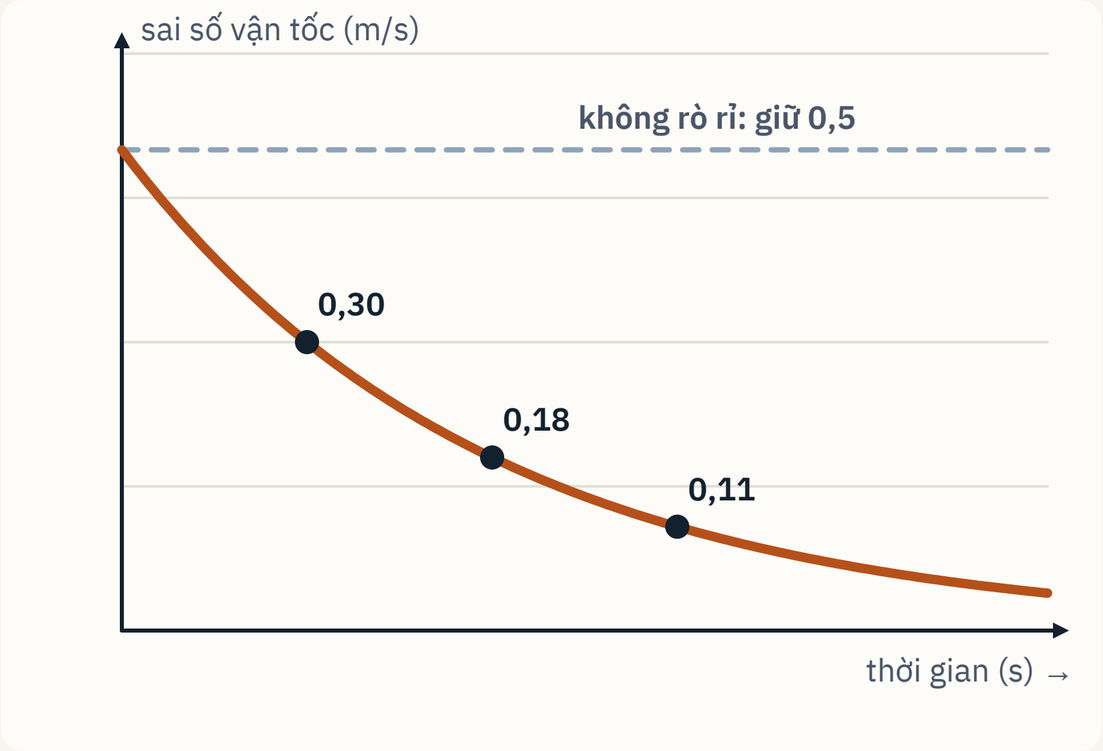
Mỗi giây, vận tốc chỉ giữ **60%** chính nó: độ lệch không thể cộng dồn mãi.

powf(0.60, dt) làm điều đó đúng theo giây bất kể tốc độ vòng (≈ 0,998 mỗi vòng).

Khi giữ yên, vận tốc được đặt về 0 — điều duy nhất drone biết chắc.

VÍ DỤ

Cái giá: di chuyển đều kéo dài bị ước tính thấp một chút — đánh đổi đúng cho thiết bị chủ yếu lơ lửng.



DẪN ĐƯỜNG

Hệ tọa độ cất cánh và NAV_CMD_ALIGN

Hệ cất cánh: +x là hướng mũi, +y ở bên trái mũi.

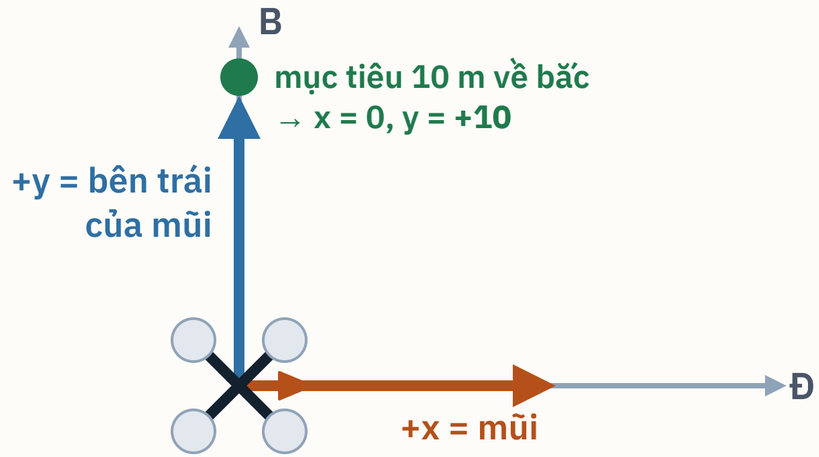
Drone không có la bàn: điện thoại gửi hướng mũi lúc cất cánh — **ALIGN**.

Mục tiêu đến dưới dạng mét về **bắc** và **đông**; hai dòng đổi chúng sang hệ cất cánh:

x = n·cos a + e·sin a, **y = n·sin a − e·cos a**.

VÍ DỤ

a = 90°, mục tiêu 10 m về bắc: x = 10·0 + 0 = 0, y = 10·1 − 0 = 10 → 10 m bên trái. ✓



Mũi hướng đông (ALIGN báo 90°), nên phía bắc ở bên trái drone.

Xoay theo drone: hai phép quay

```
const float yaw_rel = (heading_deg -
heading_ref_deg)
                      * NAV_HEADING_SIGN *
DEG_TO_RAD;
// body → launch (navUpdate)
ax_w = fwd * cy - left * sy;
ay_w = fwd * sy + left * cy;
// launch → body (navSetpoints)
body_fwd  =  ux * cy + uy * sy;
body_left = -ux * sy + uy * cy;
```

Drone có thể xoay sau khi cất cánh. Gia tốc đo trong hệ thân, còn vị trí nằm trong hệ cất cánh.

Một phép quay đi thân → cất cánh, phép kia là nghịch đảo chính xác.

NAV_HEADING_SIGN = −1 làm chiều hướng của gyro khớp với hệ dương-bên-trái.

DẪN ĐƯỜNG

navSetpoints(): nghiêng về phía mục tiêu

```
float ex = tgt_x - pos_x, ey = tgt_y - pos_y;
float dist = sqrtf(ex*ex + ey*ey);
if (dist < NAV_STOP_BAND_M || !tgt_valid) return;   // 0.6 m
float mag = constrain(dist * nav_kp, 0.0f, NAV_MAX_TILT_DEG);
float ux = ex / dist * mag;      // toward the target
float uy = ey / dist * mag;
// rotate into the body frame, then:
sp_pitch_deg = constrain(body_fwd,  -MAX_TILT_DEG, MAX_TILT_DEG);
sp_roll_deg  = constrain(body_left, -MAX_TILT_DEG, MAX_TILT_DEG);
```

Bộ điều khiển P đơn giản: 1,2° nghiêng mỗi mét (nav_kp), tối đa **12°**. Trong **0,6 m**, hoặc không có mục tiêu: giữ thăng bằng. RTH = mục tiêu (0, 0).

Ví dụ: mục tiêu 5 m phía trước → 5 × 1,2 = 6° chúc mũi. 15 m → 18 → giới hạn còn 12°.

DẪN ĐƯỜNG · HỎI TẠI SAO

Kiểm tra hiểu biết

Vì sao sai số vị trí tăng nhanh hơn sai số vận tốc?

Gợi ý: độ lệch bị tích phân mấy lần?

Vì sao đặt vận tốc về 0 khi giữ yên?

Gợi ý: lúc đó drone biết chắc điều gì?

Vì sao điện thoại gửi ALIGN lúc cất cánh?

Gợi ý: drone thiếu cảm biến nào?

Vì sao góc nghiêng dẫn đường nhỏ hơn khi lái tay?

Gợi ý: drone chắc chắn về vị trí cỡ nào?

11

Kiểm thử và quy trình làm việc

Chứng minh phần toán khó trên máy tính, rồi build, nạp và đọc dữ liệu drone.

KIỂM THỬ

Năm chương trình kiểm thử chạy trên máy tính

Bài kiểm thử	Chứng minh điều gì
frame_test	mục tiêu từ điện thoại nằm đúng hệ mà ước tính vị trí dùng
yawrot_test	phép quay thân ↔ cất cánh quay đúng chiều, với dấu của sketch
logcfg_test	cài đặt khối log vẫn đúng khi bị gửi lại
level_test	tự cân bằng kéo về mục tiêu và không bao giờ lật
sbus_test	khung mất và failsafe bị loại; khung tốt được nhận

Chạy tất cả bằng: **sh Esp32_drone/test/run.sh** — khoảng 100 dòng “ok” khi mọi thứ đạt.

KIỂM THỬ

Các kiểm tra trong run.sh: bài kiểm thử không thể lỗi thời

```
check() {
  if ! grep -qF "$1" "$sketch"; then
    echo "STALE TEST: '$1' is no longer in
Esp32_drone.ino"
    exit 1
  fi
}
check 'roll_setpoint  = sp_roll_rate  -
roll_level_adjust;'
check 'if (f[23] & 0x08) return false;'
check 'tgt_y = n * sa - e * ca;'
```

Bài kiểm thử chép công thức từ sketch, vì sketch không biên dịch được trên máy tính.

Bản chép có thể lỗi thời âm thầm. **check** báo lỗi to nếu đúng dòng đó bị đổi.

Kiểm tra thêm: giới hạn chế độ, thang gia tốc kế, NAV_HEADING_SIGN.

KIỂM THỬ

Bên trong level_test: một mô phỏng nhỏ

```
static float simulate(float angle0, float sp_deg,
                      float sign, float seconds) {
  float angle = angle0;
  for (int i = 0; i < (int)(seconds / 0.004f); i++) {
    float err = angle - sp_deg;
    float adj = clampf(err * 15.0f, -200, 200);
    float gyro = 0.0f + sign * adj;  // perfect rate loop
    angle += gyro * 0.004f;
    if (fabsf(angle) > 180.0f) return angle;   // flipped
  }
  return angle;
}
```

Mô phỏng một trục ở 250 Hz với vòng tốc độ hoàn hảo.

sign −1 = firmware hiện tại → góc co về mục tiêu.

sign +1 = lỗi cũ → góc tăng và lật.

VÍ DỤ

Bắt đầu ở +10°, mục tiêu 0°: sau 1 s với sign −1 → khoảng 0°. Với sign +1 → vượt 180°, lật.

KIỂM THỬ

sbus_test kiểm tra những gì

Trường hợp	Byte cờ 23	Byte cuối	Mong đợi
khung bình thường	0x00	0x00	nhận
bật bit kênh số 17/18	0x03	0x00	nhận
byte cuối SBUS2	0x00	0x14	nhận
máy thu failsafe	0x08	0x00	loại
mất khung	0x04	0x00	loại
byte cuối lệch / đầu khung sai	—	0x55 / —	loại
kiểm tra byte 24 cũ (đối chứng âm)	0x08	0x00	nhận sai

KIỂM THỬ

Tự thêm một bài kiểm thử

1. Chọn một công thức hay quy tắc sắp đổi, và chép các dòng đó vào một file .cpp kiểm thử mới.
2. Viết các trường hợp có đáp án biết trước, tính tay: một trường hợp tốt và một trường hợp xấu.
3. Giữ phiên bản cũ làm **đối chứng âm** để thấy bài kiểm thử có thể thất bại.
4. Thêm dòng **check '...'** vào run.sh cho mỗi dòng đã chép, và một bước build-và-chạy.
5. Chạy **sh run.sh**, rồi sửa sketch, rồi chạy lại.

KIỂM THỬ

Cổng serial trên bàn

Lệnh	Làm gì
status	kết nối, chế độ, trạng thái auto, armed, lực đẩy hover, hướng, pin
save / load / defaults	hệ số PID + cài đặt bay trong flash
link n	đổi cấu hình kết nối 0–5, rồi khởi động lại
sta ssid pass	lưu thông tin Wi-Fi station

Telemetry, mỗi giây một dòng: **disarm | VBAT:3.92V | RC r:1500 p:1500 t:1000 y:1500 | ang p:+0.4 r:-0.2 | mot RF:1000 …**

KIỂM THỬ

Build và nạp firmware

1. Cài gói board **esp32 by Espressif** trong Arduino IDE 2.x (hoặc arduino-cli).
2. Mở **Esp32_drone/Esp32_drone.ino**. Board: **ESP32 Dev Module**.
3. Partition scheme: **Huge APP** — bắt buộc, radio cần khoảng 1,7 MB.
4. Upload, rồi mở serial monitor ở **115200**.
5. Giữ drone **phẳng và đứng yên** khi có thông báo hiệu chuẩn gyro.

TỔNG KẾT

Lộ trình học: năm bài tập

1. **Đọc** loop() và ghi lại bước đánh số nào thay đổi từng biến toàn cục.

2. **Trên bàn**: tháo cánh, arm, nghiêng drone bằng tay và xem động cơ nào tăng tốc.

3. **Log**: vẽ pid.roll_p, roll_i và roll_d trong ứng dụng khi lơ lửng. Thành phần nào làm việc chính?

4. **Kiểm thử**: cố ý làm sai dấu tự cân bằng và xem level_test thất bại. Rồi sửa lại.

5. **Mở rộng**: đổi D sang theo phép đo, hoặc thêm mixer scaling — viết kiểm thử trước.

TỔNG KẾT

Thuật ngữ

Từ	Nghĩa	Từ	Nghĩa
Setpoint	điều ta muốn	Bias (độ lệch)	sai số cảm biến khi đứng yên
Error (sai số)	đo được – mong muốn	Wind-up	I tăng khi không tác động được
Rate	tốc độ xoay, °/s	Failsafe	dừng khi mất đầu vào
Attitude	các góc nghiêng	Idempotent	áp dụng hai lần vẫn an toàn
Duty	tỷ lệ thời gian BẬT	Dead reckoning	vị trí chỉ từ chuyển động
CRTP	giao thức điện thoại	TOC	danh sách tham số / biến log

TỔNG KẾT · HỎI TẠI SAO

Những câu hỏi sâu nên hỏi mãi

Vì sao mọi lỗi dấu trong firmware này đều làm lật, không chỉ lắc?

Gợi ý: phản hồi dương tăng không giới hạn.

Vì sao mỗi cảm biến thiếu tạo đúng một giới hạn ở một chế độ?

Gợi ý: ghép áp suất, la bàn, GPS, khoảng cách với HOLD, hướng, FOLLOW, hạ cánh.

Vì sao phần lớn kiểm tra an toàn nằm ở rìa của mã?

Gợi ý: dữ liệu không tin cậy đi vào ở đâu?

Vì sao giữ phiên bản lỗi trong mỗi bài kiểm thử?

Gợi ý: làm sao biết bài kiểm thử có thể thất bại?

TỔNG KẾT

Nguồn tham khảo

Nguồn	Dùng cho
Esp32_drone/Esp32_drone.ino	mọi slide mã, giá trị và chú thích được trích
Esp32_drone/test/*.cpp, run.sh	phần kiểm thử và các câu chuyện lỗi
docs/INSTALL.md, docs/USER_GUIDE.md	các bước build, CẤT ĐỘNG CƠ, quy tắc an toàn
Datasheet + bản đồ thanh ghi TDK InvenSense MPU-6500	dải cảm biến và thanh ghi
Datasheet + Technical Reference Espressif ESP32 (LEDC)	tính năng chip, xung PWM
Tài liệu CRTP của Bitcraze	bố cục giao thức điện thoại
Datasheet Alpha & Omega AO3416	MOSFET động cơ